

Anexos metodológicos: Código fuente desarrollado y figuras

Anexo I: SCRIPT PROCESADO DE PUNTOS

GBIF:

```
#
=====
# SCRIPT: Limpieza de CSVs individuales por
especie para Biomod2
#       Península Ibérica – TFG 2026
#
-----
# Descripción: Lee cada CSV individual de la
carpeta de especies (separados
#             por tabulaciones, formato GBIF)
y aplica:
#             1) Selección de columnas
necesarias
#             2) Eliminación de filas sin
coordenadas
#             3) CoordinateCleaner (outliers
espaciales)
#             4) Spatial thinning a 1 km
(sesgo de muestreo)
#             Sobreescribe cada archivo con la
versión limpia final.
#
=====
#
=====
# 0. INSTALACIÓN Y CARGA DE LIBRERÍAS
#
=====
=====

paquetes_necesarios <- c(
  "CoordinateCleaner",
  "dplyr",
  "stringr"
)

paquetes_a_instalar <- paquetes_necesarios[
  !paquetes_necesarios %in%
installed.packages()[, "Package"]
]

if (length(paquetes_a_instalar) > 0) {
  install.packages(paquetes_a_instalar,
dependencies = TRUE)
}

invisible(lapply(paquetes_necesarios, library,
character.only = TRUE))

message("✓ Librerías cargadas.")

#
=====
# 1. PARÁMETROS Y RUTAS
```

```
#
=====
# — Carpeta con los CSVs individuales por
especie
RUTA_CARPETA <-
r"(E:\TFG\2026\DATOS\CORRECCION\ESPECIES\nueva
correccion)"

# — Resolución del thinning
=====
RESOLUCION_GRADOS <- 1 / 120 # ~1 km

# — Mínimo de puntos para modelado en Biomod2
=====
MIN_PUNTOS <- 30

# — Listar CSVs de la carpeta (excluyendo el
log de resumen)
archivos_csv <- list.files(RUTA_CARPETA,
pattern = "\\\.csv$", full.names = TRUE)
archivos_csv <-
archivos_csv[!basename(archivos_csv) %in%
"00_resumen_limpieza.csv"]

if (length(archivos_csv) == 0) {
  stop(" No se encontraron archivos CSV en: ",
RUTA_CARPETA)
}

message("✓ ", length(archivos_csv), " archivos
CSV detectados.")

#
=====
# 2. FUNCIÓN DE SPATIAL THINNING A 1 KM
#
=====

thinning_1km <- function(df, res_grados = 1 /
120) {

  df$celda_id <- paste(
    floor(df$decimalLongitude / res_grados),
    floor(df$decimalLatitude / res_grados),
    sep = "_"
  )

  set.seed(42)
  df <- df[sample(nrow(df)), ]
  df <- df[!duplicated(df$celda_id), ]
  df$celda_id <- NULL

  return(df)
}

#
=====
```

```

# 3. BUCLE PRINCIPAL: PROCESAR CADA CSV
#
=====

message("\n— Iniciando procesado...\n")

resumen_final <- data.frame(
  archivo      = character(),
  n_entrada    = integer(),
  n_con_coords = integer(),
  n_tras_cc    = integer(),
  n_final      = integer(),
  apto_biomod2 = character(),
  stringsAsFactors = FALSE
)

for (ruta_csv in archivos_csv) {

  nombre_archivo <- basename(ruta_csv)
  message("► Procesando: ", nombre_archivo)

  # — Leer CSV (separado por tabulaciones,
  formato GBIF)
  datos <- tryCatch(
    read.csv(ruta_csv, stringsAsFactors =
FALSE, sep = "\t"),
    error = function(e) {
      message(" Error al leer: ", e$message,
"\n")
      return(NULL)
    }
  )

  if (is.null(datos) || nrow(datos) < 5) {
    message(" ⚠ Menos de 5 registros. Archivo
omitido.\n")
    next
  }

  n_entrada <- nrow(datos)
  message(" Registros entrada:      ",
n_entrada)

  # — Seleccionar solo columnas necesarias
  # El CSV de GBIF tiene decenas de columnas;
  solo necesitamos estas cuatro.
  cols_necesarias <- c("species",
"decimalLatitude", "decimalLongitude", "year")

  cols_faltantes <-
cols_necesarias[!cols_necesarias %in%
names(datos)]
  if (length(cols_faltantes) > 0) {
    message(" Columnas no encontradas: ",
paste(cols_faltantes, collapse = ", "))
    message(" Columnas disponibles: ",
paste(names(datos), collapse = ", "), "\n")
    next
  }

  datos <- datos[, cols_necesarias]

  # — Eliminar filas sin coordenadas

```

```

# CoordinateCleaner falla si encuentra NAs en
lat/lon.
  datos <- datos[!is.na(datos$decimalLatitude)
&
!is.na(datos$decimalLongitude), ]

  n_con_coords <- nrow(datos)
  message(" Registros con coordenadas:      ",
n_con_coords)

  if (n_con_coords < 5) {
    message(" ⚠ Menos de 5 registros con
coordenadas. Archivo omitido.\n")
    next
  }

  # — CoordinateCleaner
  # Elimina coordenadas (0,0), centroides de
  países/provincias, capitales,
  # sedes de instituciones científicas y puntos
  en el mar.
  datos_cc <- tryCatch({
    cc <- CoordinateCleaner::clean_coordinates(
      x      = datos,
      lon    = "decimalLongitude",
      lat    = "decimalLatitude",
      species = "species",
      tests  = c("zeros", "equal", "capitals",
"centroids",
"gbif", "institutions",
"seas"),
      value  = "spatialvalid"
    )
    cc[cc$.summary == TRUE, !grepl("^\\.",
names(cc))]
  }, error = function(e) {
    message(" ⚠ Error en CoordinateCleaner:
", e$message)
    return(datos)
  })

  n_tras_cc <- nrow(datos_cc)
  message(" Registros tras CoordCleaner:      ",
n_tras_cc)

  # — Spatial thinning a 1 km
  # Un único registro por celda de ~1 km para
  corregir el sesgo de muestreo
  # generado por plataformas de ciencia
  ciudadana (iNaturalist, eBird).
  datos_final <- thinning_1km(datos_cc,
res_grados = RESOLUCION_GRADOS)

  n_final <- nrow(datos_final)
  message(" Registros tras thinning 1 km:      ",
n_final)

  # — Aviso si hay pocos puntos para Biomod2
  apto <- ifelse(n_final >= MIN_PUNTOS, "Sí",
"NO – revisar")
  if (n_final < MIN_PUNTOS) {

```

```

        message("  △ ATENCIÓN: solo ", n_final,
              " puntos. Especie NO apta para
Biomod2.")
    }

    # — Guardar CSV limpio (sobreescribe el
original) —————
    write.csv(datos_final, file = ruta_csv,
row.names = FALSE)
    message("  ✓ Guardado: ", nombre_archivo,
"\n")

    resumen_final <- rbind(resumen_final,
data.frame(
        archivo      = nombre_archivo,
        n_entrada    = n_entrada,
        n_con_coords = n_con_coords,
        n_tras_cc    = n_tras_cc,
        n_final      = n_final,
        apto_biomod2  = apto
    ))
}

#
=====
# 4. RESUMEN FINAL
#
=====

message(paste(rep("=", 70), collapse = ""))
message("RESUMEN DEL PROCESADO")
message(paste(rep("=", 70), collapse = ""))
print(resumen_final, row.names = FALSE)

no_aptas <-
resumen_final[resumen_final$apto_biomod2 !=
"Sí", "archivo"]
if (length(no_aptas) > 0) {
    message("\n△ Especies con menos de ",
MIN_PUNTOS,
        " puntos (revisar antes de
Biomod2):")
    print(no_aptas)
}

ruta_log <- file.path(RUTA_CARPETA,
"00_resumen_limpieza.csv")
write.csv(resumen_final, file = ruta_log,
row.names = FALSE)

message("\n✓ Log guardado en: ", ruta_log)
message("✓ Script finalizado. Archivos en: ",
RUTA_CARPETA)

```

Anexo II: SCRIPT de CONSOLIDACIÓN DE SHAPEFILES DE CUADRÍCULAS (1KM) A PUNTOS

```

library(terra)
# 1. CONFIGURACIÓN DE RUTAS Y PARÁMETROS
RUTA_ESPECIES <-
r"(E:\TFG\2026\DATOS\CORRECCION\ESPECIES\nueva
correccion)"

```

```

RUTA_RASTER_REF <-
r"(E:\TFG\2026\DATOS\CORRECCION\VARIABLES
PREDICTORAS\PROCESADAS\LIPIAS_PARA_MAXENT\RESU
MEN_FINAL\BIO04_MEAN.tif)"
RUTA_SALIDA_CSV <-
r"(E:\TFG\2026\DATOS\CORRECCION\ESPECIES\presen
cias_finales_maxent.csv)"

# Categorías de amenaza (nombres exactos de tus
carpetas principales)
categorias <- c("ENDANGERED", "VULNERABLE",
"NEAR THREATENED", "LEAST CONCERN")

# Cargar el ráster de referencia para clonar su
Sistema de Coordenadas (CRS)
r_ref <- rast(RUTA_RASTER_REF)
crs_destino <- crs(r_ref)

# Dataframe maestro donde unificaremos todas
las especies
df_maestro <- data.frame(species = character(),
X = numeric(), Y = numeric(), stringsAsFactors
= FALSE)

cat("=== Iniciando la extracción de centroides
desde los .shp oficiales ===\n")

# 2. BUCLE RECURSIVO POR CATEGORÍAS Y CARPETAS
DE ESPECIE
for (cat_amenaza in categorias) {
    ruta_cat <- paste0(RUTA_ESPECIES, "/",
cat_amenaza)

    if (!dir.exists(ruta_cat)) next # Si una
carpeta no existe, la salta

    # Listar las subcarpetas de las especies
dentro de esta categoría
    carpetas_especies <- list.dirs(ruta_cat,
full.names = FALSE, recursive = FALSE)

    for (especie in carpetas_especies) {
        ruta_folder_especie <- paste0(ruta_cat,
"/", especie)

        # Buscar el archivo .shp dentro de la
carpeta de la especie
        archivo_shp <-
list.files(ruta_folder_especie, pattern =
"\\.shp$", full.names = TRUE)

        if (length(archivo_shp) == 0) {
            cat(" [AVISO]: No se encontró archivo
.shp en la carpeta de:", especie, "\n")
            next
        }

        cat("-> Procesando malla de:", especie,
"\n")

        # 1. Leer las cuadrículas de 1km de la
especie
        cuadrículas <- vect(archivo_shp[1])

        # 2. Reproyectar las cuadrículas al CRS
exacto de tus mapas climáticos

```

```

    cuadrículas_proyectadas <-
project(cuadrículas, crs_destino)

    # 3. Calcular el centroide (punto central)
de cada cuadrado de 1km
    puntos_centroides <-
centroids(cuadrículas_proyectadas)

    # 4. Extraer las coordenadas numéricas X e
Y ya alineadas con el clima
    coordenadas <- crds(puntos_centroides)

    # 5. Crear la tabla limpia para esta
especie
    df_especie <- data.frame(
        species = especie, # Usa el nombre de la
carpeta como nombre limpio de la especie
        X = coordenadas[, 1],
        Y = coordenadas[, 2],
        stringsAsFactors = FALSE
    )

    # Acumular en la tabla maestra
    df_maestro <- rbind(df_maestro, df_especie)
}
}

# 3. EXPORTAR EL ARCHIVO MAESTRO PARA MAXENT /
BIOMOD2
write.csv(df_maestro, file = RUTA_SALIDA_CSV,
row.names = FALSE)

cat("\n=====
=====\\n")
cat("✓ ¡PROCESO COMPLETADO CON ÉXITO!\\n")
cat("✓ Archivo maestro generado en:",
RUTA_SALIDA_CSV, "\\n")
cat("✓ Total de celdas de 1km recopiladas
entre todas las especies:", nrow(df_maestro),
"\\n")
cat("=====
=====\\n")

```


Anexo III: Tabla explicativa de variables bioclimáticas

Variable	Abreviatura	Nombre / significado	Descripción
Temperatura media anual	BIO1	Annual Mean Temperature	Temperatura media de un año, representativa de las condiciones térmicas generales del área.
Rango medio diurno	BIO2	Mean Diurnal Range	Diferencia media entre la temperatura máxima y mínima diaria, indicando la amplitud térmica diaria.
Isothermalidad	BIO3	Isothermality	Relación entre el rango medio diurno y el rango anual de temperatura. Indica la importancia relativa de las variaciones diarias frente a las anuales.
Estacionalidad de la temperatura	BIO4	Temperature Seasonality	Variabilidad anual de la temperatura, expresada como desviación estándar $\times 100$. Valores elevados indican mayor estacionalidad térmica.
Temperatura máxima del mes más cálido	BIO5	Max Temperature of Warmest Month	Temperatura máxima registrada durante el mes más cálido del año.
Temperatura mínima del mes más frío	BIO6	Min Temperature of Coldest Month	Temperatura mínima registrada durante el mes más frío del año.
Rango anual de temperatura	BIO7	Temperature Annual Range	Diferencia entre BIO5 y BIO6, representando la amplitud térmica anual.
Temperatura media del trimestre más húmedo	BIO8	Mean Temperature of Wettest Quarter	Temperatura media del trimestre de tres meses consecutivos con mayor precipitación acumulada.
Temperatura media del trimestre más seco	BIO9	Mean Temperature of Driest Quarter	Temperatura media del trimestre de tres meses consecutivos con menor precipitación acumulada.
Temperatura media del trimestre más cálido	BIO10	Mean Temperature of Warmest Quarter	Temperatura media del trimestre de tres meses consecutivos con mayor temperatura media.
Temperatura media del trimestre más frío	BIO11	Mean Temperature of Coldest Quarter	Temperatura media del trimestre de tres meses consecutivos con menor temperatura media.
Precipitación anual	BIO12	Annual Precipitation	Precipitación total acumulada durante el año.
Precipitación del mes más lluvioso	BIO13	Precipitation of Wettest Month	Precipitación acumulada durante el mes con mayor precipitación.
Precipitación del mes más seco	BIO14	Precipitation of Driest Month	Precipitación acumulada durante el mes con menor precipitación.
Estacionalidad de la precipitación	BIO15	Precipitation Seasonality	Variabilidad de la precipitación a lo largo del año, expresada mediante el coeficiente de variación.

Precipitación del trimestre más húmedo	BIO16	Precipitation of Wettest Quarter	Precipitación acumulada durante el trimestre de tres meses consecutivos más húmedo.
Precipitación del trimestre más seco	BIO17	Precipitation of Driest Quarter	Precipitación acumulada durante el trimestre de tres meses consecutivos más seco.
Precipitación del trimestre más cálido	BIO18	Precipitation of Warmest Quarter	Precipitación acumulada durante el trimestre de tres meses consecutivos con mayor temperatura media.
Precipitación del trimestre más frío	BIO19	Precipitation of Coldest Quarter	Precipitación acumulada durante el trimestre de tres meses consecutivos con menor temperatura media.
Índice de Vegetación de Diferencia Normalizada	NDVI	Normalized Difference Vegetation Index	Índice que permite estimar la presencia, densidad y actividad de la vegetación a partir de la respuesta espectral de la superficie.
Índice de Vegetación Mejorado	EVI	Enhanced Vegetation Index	Índice de vegetación diseñado para mejorar la sensibilidad en zonas de vegetación densa y reducir determinadas influencias atmosféricas y del suelo.
Temperatura superficial terrestre	LST	Land Surface Temperature	Temperatura de la superficie terrestre, que representa las condiciones térmicas de la superficie y no directamente la temperatura del aire.
Humedad del suelo	SM	Soil Moisture	Contenido de agua estimado en el suelo, utilizado como indicador de disponibilidad hídrica para la vegetación y los ecosistemas.

Anexo IV: Script en Python para la Descarga Virtual Diaria, Agregación Mensual y Cálculo Bioclimático (2001–2025)

```
import sys
import logging
import warnings
import calendar
from pathlib import Path
import numpy as np
import rasterio
from rasterio.windows import from_bounds
from tqdm import tqdm

warnings.filterwarnings("ignore")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s]
%(message)s",
```

```
handlers=[
    logging.StreamHandler(sys.stdout),
    logging.FileHandler("chelsa_processing.log"),
],
)
log = logging.getLogger(__name__)

# — CONFIGURACIÓN DEL ESPACIO DE TRABAJO

YEARS_MONTHLY = list(range(2001, 2022))
YEARS_DAILY = list(range(2022, 2026))
VARIABLES = ["tas", "tasmin", "tasmx", "pr"]
BBOX = (-9.5, 35.9, 3.4, 43.8)

BASE_DIR = Path("chelsa_data")
BIO_DIR = BASE_DIR / "bioclim"
BIO_DIR.mkdir(parents=True, exist_ok=True)

CHELSA_MONTHLY =
"https://os.unil.cloud.switch.ch/chelsa02/chelsa/global/monthly"
```

```

CHELSA_DAILY =
"https://os.unil.cloud.switch.ch/chelsa02/chelsa/global/daily"

SCALE = {"tas": 0.1, "tasmin": 0.1, "tasmax":
0.1, "pr": 0.1}
OFFSET = {"tas": -273.15, "tasmin": -273.15,
"tasmax": -273.15, "pr": 0.0}

def build_monthly_url(var, year, month):
    return
    f"{CHELSA_MONTHLY}/{var}/{year}/CHELSA_{var}_{month:02d}_{year}_V.2.1.tif"

def build_daily_url(var, year, month, day):
    return
    f"{CHELSA_DAILY}/{var}/{year}/CHELSA_{var}_{month:02d}_{day:02d}_{year}_V.2.1.tif"

def load_iberia(url):
    try:
        with rasterio.open(f"/vsicurl/{url}")
as src:
        window = from_bounds(*BBOX,
src.transform)
        data = src.read(1,
window=window).astype("float32")
        transform =
src.window_transform(window)
        crs = src.crs
        if src.nodata is not None:
            data[data == src.nodata] =
np.nan
        return data, transform, crs
    except:
        return None, None, None

def load_monthly_year(year):
    result = {}
    transform, crs = None, None
    for var in VARIABLES:
        months = []
        for m in range(1, 13):
            data, tr, c =
load_iberia(build_monthly_url(var, year, m))
            if data is None:
                return None
            months.append(data)
            if transform is None:
                transform, crs = tr, c
        arr = np.stack(months)
        arr = arr * SCALE[var] + OFFSET[var]
        result[var] = arr
        result["_transform"] = transform
        result["_crs"] = crs
    return result

def daily_to_monthly(year):
    acc = {v: [[] for _ in range(12)] for v in
VARIABLES}
    transform, crs, shape = None, None, None

    for m in range(1, 13):
        _, ndays = calendar.monthrange(year, m)
        for d in tqdm(range(1, ndays+1),
desc=f"Descarga Virtual {year}-{m:02d}"):
            for var in VARIABLES:

```

```

                data, tr, c =
load_iberia(build_daily_url(var, year, m, d))
                if data is not None:
                    acc[var][m-1].append(data)
                    if transform is None:
                        transform, crs, shape =
tr, c, data.shape

                if transform is None:
                    return None

            monthly = {}
            for var in VARIABLES:
                arr = np.zeros((12,) + shape,
dtype=np.float32)
                for m in range(12):
                    if acc[var][m]:
                        stack = np.stack(acc[var][m])
                        if var == "pr":
                            arr[m] = np.nansum(stack,
axis=0)
                        else:
                            arr[m] = np.nanmean(stack,
axis=0)
                    else:
                        arr[m] = np.nan
                arr = arr * SCALE[var] + OFFSET[var]
                monthly[var] = arr

            monthly["_transform"] = transform
            monthly["_crs"] = crs
            return monthly

def quarter(values, criteria, mode, stat):
    qcrit = np.array([criteria[i%12] +
criteria[(i+1)%12] + criteria[(i+2)%12] for i
in range(12)])
    qvals = np.array([values[i%12] +
values[(i+1)%12] + values[(i+2)%12] for i in
range(12)])
    idx = np.argmax(qcrit, axis=0) if mode in
("wettest", "warmest") else np.argmin(qcrit,
axis=0)
    lat, lon = np.indices(values.shape[1:])
    res = qvals[idx, lat, lon]
    return res if stat == "sum" else res / 3.0

def compute_bioclim(m):
    t, tmin, tmax, pr = m["tas"], m["tasmin"],
m["tasmax"], m["pr"]
    bio = {}
    bio["BI001"] = np.nanmean(t, axis=0)
    bio["BI002"] = np.nanmean(tmax - tmin,
axis=0)
    tr = np.nanmax(t, axis=0) - np.nanmin(t,
axis=0)
    bio["BI003"] = np.where(tr > 0,
bio["BI002"]/tr*100, np.nan)
    bio["BI004"] = np.nanstd(t, axis=0)*100
    bio["BI005"] = np.nanmax(tmax, axis=0)
    bio["BI006"] = np.nanmin(tmin, axis=0)
    bio["BI007"] = bio["BI005"] - bio["BI006"]
    bio["BI008"] = quarter(t, pr, "wettest",
"mean")
    bio["BI009"] = quarter(t, pr, "driest",
"mean")

```

```

        bio["BI010"] = quarter(t, t, "warmest",
"mean")
        bio["BI011"] = quarter(t, t, "coldest",
"mean")
        bio["BI012"] = np.nansum(pr, axis=0)
        bio["BI013"] = np.nanmax(pr, axis=0)
        bio["BI014"] = np.nanmin(pr, axis=0)
        pmean = np.nanmean(pr, axis=0)
        bio["BI015"] = np.where(pmean > 0,
np.nanstd(pr, axis=0)/pmean*100, 0)
        bio["BI016"] = quarter(pr, pr, "wettest",
"sum")
        bio["BI017"] = quarter(pr, pr, "driest",
"sum")
        bio["BI018"] = quarter(pr, t, "warmest",
"sum")
        bio["BI019"] = quarter(pr, t, "coldest",
"sum")
        return bio

def save_bios(bio, transform, crs, year):
    for key, data in bio.items():
        out = BIO_DIR /
f"{key}_iberica_{year}.tif"
        with rasterio.open(
            out, "w", driver="GTiff",
            height=data.shape[0], width=data.shape[1],
            count=1, dtype="float32", crs=crs,
            transform=transform,
            compress="lzw", nodata=np.nan
        ) as dst:
            dst.write(data.astype("float32"),
1)
            log.info(f"Año {year} procesado (19
variables).")

def process_year(year):
    if (BIO_DIR /
f"BI001_iberica_{year}.tif").exists():
        log.info(f"El año {year} ya existe.")
        return
    m = load_monthly_year(year) if year <= 2021
else daily_to_monthly(year)
    if m is None:
        log.warning(f"Fallo en el año: {year}")
        return
    bio = compute_bioclim(m)
    save_bios(bio, m["_transform"], m["_crs"],
year)

def main():
    for y in YEARS_MONTHLY + YEARS_DAILY:
        process_year(y)
    log.info("=== PROCESO COMPLETADO ===")

if __name__ == "__main__":
    main()

```

Anexo V: Script en Google Earth Engine (JavaScript) para la Extracción, Escalamiento y Reducción Anual de Coberturas MODIS y TerraClimate (2001–2025)

```

//
=====
// TFG - SERIE COMPLETA COMPILADA DE VARIABLES
SATELITALES E HIDROLÓGICAS (2001-2025)
// UNIDAD DE ANÁLISIS: PENÍNSULA IBÉRICA
CONTINENTAL (EXCLUSIÓN DE ARCHIPIÉLAGOS)
//
=====

// DEFINICIÓN DE LÍMITES
POLÍTICO-ADMINISTRATIVOS (MÁSCARA CONTINENTAL
DE REFERENCIA)
var level1 =
ee.FeatureCollection("FAO/GAUL/2015/level1");

// Aislamiento de la masa continental de España
mediante exclusión de archipiélagos
var spainPeninsula =
level1.filter(ee.Filter.eq('ADM0_NAME',
'Spain'))
        .filter(ee.Filter.neq('ADM1_NAME', 'Islas
Balears'))
        .filter(ee.Filter.neq('ADM1_NAME',
'Canarias'));

// Aislamiento de Portugal continental mediante
exclusión de islas ultraperiféricas
var portugalPeninsula =
level1.filter(ee.Filter.eq('ADM0_NAME',
'Portugal'))
        .filter(ee.Filter.neq('ADM1_NAME', 'Açores'))
        .filter(ee.Filter.neq('ADM1_NAME',
'Madeira'));

// Fusión de geometrías para consolidar la
Región de Interés (ROI) Ibérica
var iberia =
spainPeninsula.merge(portugalPeninsula);

// FUNCIÓN MATRIZ PARA LA EXPORTACIÓN
AUTOMATIZADA A GOOGLE DRIVE
function exportVariable(image, year, varName) {
    Export.image.toDrive({
        image: image.float(),
        description: varName + '_1km_Peninsula_' +
year,
        folder: 'TFG_VARIABLES_PENINSULA',
        fileNamePrefix: varName + '_1km_Peninsula_'
+ year,
        region: iberia.geometry(),
        scale: 1000, // Forzado de resolución
espacial nominal a 1 km
        crs: 'EPSG:4326', // Sistema de Coordenadas
Geográficas Nativo
        maxPixels: 1e13
    });
}

// MÓDULOS DE EXTRACCIÓN MODAL Y CALIBRACIÓN
RADIOMÉTRICA

// Extracción y escalamiento del Índice de
Vegetación de Diferencia Normalizada (NDVI)
function exportNDVI(year) {

```

```

    var img =
ee.ImageCollection("MODIS/061/MOD13Q1")
    .filterDate(year + '-01-01', year
+ '-12-31')
    .select('NDVI')
    .mean()
    .multiply(0.0001) // Factor de
escala para restituir valores reales (-1 a 1)
    .clip(iberia);
    exportVariable(img, year, 'NDVI');
}

// Extracción y escalamiento del Índice de
Vegetación Mejorado (EVI)
function exportEVI(year) {
    var img =
ee.ImageCollection("MODIS/061/MOD13Q1")
    .filterDate(year + '-01-01', year
+ '-12-31')
    .select('EVI')
    .mean()
    .multiply(0.0001) // Factor de
escala idéntico al NDVI
    .clip(iberia);
    exportVariable(img, year, 'EVI');
}

// Extracción, calibración y conversión de la
Temperatura Superficial Terrestre (LST)
function exportLST(year) {
    var img =
ee.ImageCollection("MODIS/061/MOD11A2")
    .filterDate(year + '-01-01', year
+ '-12-31')
    .select('LST_Day_1km')
    .mean()
    .multiply(0.02) // Factor de
escala original del producto MODIS
    .subtract(273.15) //
Transformación matemática de Kelvin a grados
Celsius
    .clip(iberia);
    exportVariable(img, year, 'LST');
}

// Extracción y promedio de la Humedad del
Suelo (Balance Hídrico - TerraClimate)
function exportSoilMoisture(year) {
    var img =
ee.ImageCollection("IDAHO_EPSCOR/TERRACLIMATE")
    .filterDate(year + '-01-01', year
+ '-12-31')
    .select('soil') // Almacenamiento
hídrico del suelo en mm de lámina de agua
    .mean()
    .clip(iberia);
    exportVariable(img, year, 'SM');
}

// BUCLE CRONOLÓGICO ITERATIVO DE ORQUESTACIÓN
DE TAREAS (2001-2025)
var anos = [];
for (var i = 2001; i <= 2025; i++) {
    anos.push(i);
}

anos.forEach(function(y){

```

```

    exportNDVI(y);
    exportEVI(y); // <- ¡Corregido e incorporado
aquí!
    exportLST(y);
    exportSoilMoisture(y);
});

```

Anexo VI: Script en R para la Homogeneización Espacial, Remuestreo Bilineal y Máscara Continental Definitiva

```

=====
# PIPELINE EN R: HOMOGENEIZACIÓN GEOMÉTRICA Y
ARMONIZACIÓN DE EXTENSIONES RÁSTER
# PROYECTO: MODELACIÓN MULTITEMPORAL DE NICHOS
ECOLÓGICOS EN BIOMOD2
#
=====

library(terra)

# 1. CONFIGURACIÓN E INDEXACIÓN DE DIRECTORIOS
DE TRABAJO
base_dir <-
"E:/TFG/2026/DATOS/CORRECCION/VARIABLES
PREDICTORAS"
ruta_molde <- file.path(base_dir,
"PROCESADAS/NDVI_NDVI_IB_2001.tif")
ruta_nuevos <- file.path(base_dir,
"variablesmayo")
ruta_salida <- file.path(base_dir,
"PROCESADAS/LIMPIAS_PARA_MAXENT")

# Garantizar de forma preventiva la existencia
del directorio de almacenamiento
if (!dir.exists(ruta_salida)) {
    dir.create(ruta_salida, recursive = TRUE,
showWarnings = FALSE)
}

# 2. CARGA DEL MOLDE GEOMÉTRICO (PATRÓN DE
EXTENSIÓN Y ALINEACIÓN DE MALLA)
molde <- terra::rast(ruta_molde)
cat("--- Estructura del molde geométrico de
referencia cargada correctamente ---\n")
print(molde)

# 3. COMPILACIÓN DE COBERTURAS RÁSTER GENERADAS
DESDE GEE
archivos_nuevos <- list.files(ruta_nuevos,
pattern = "\\*.tif$", full.names = TRUE)

if (length(archivos_nuevos) == 0) {
    stop("ALERTA: No se localizaron archivos
GeoTIFF válidos en el directorio de entrada.")
}

# 4. PROCESAMIENTO SECUENCIAL DE ARMONIZACIÓN
TOPOLÓGICA Y ESPACIAL
for (f in archivos_nuevos) {
    cat("\nProcesando predictor estructural:",
basename(f), "\n")

```

```

# Carga de la capa multibanda / monobanda
temporal
img <- terra::rast(f)

# Fase A: Remuestreo mediante interpolación
bilineal para equilibrar tamaños de pixel
img_resampled <- terra::resample(img, molde,
method = "bilinear")

# Fase B: Enmascaramiento topológico estricto
basado en la silueta continental ibérica
img_clean <- terra::mask(img_resampled,
molde)

# Configuración de nomenclatura estructurada
de salida
nombre_salida <- file.path(ruta_salida,
paste0("CLEAN_", basename(f)))

# Fase C: Almacenamiento físico optimizado
con compresión LZW y Float32
terra::writeRaster(
img_clean,
filename = nombre_salida,
overwrite = TRUE,
datatype = "FLT4S",
gdal = "COMPRESS=LZW"
)

cat("-> Archivo optimizado y validado
exportado exitosamente en:",
basename(nombre_salida), "\n")
}

cat("\n=====
=====
cat("=== PROCESO DE ARMONIZACIÓN
TOTALMENTE COMPLETADO ===\n")
cat("Todas las variables ambientales se
encuentran listas para MaxEnt en:",
ruta_salida, "\n")
cat("=====
=====

```

Anexo VII: Script en R para la Reproyección Cartográfica, Enmascaramiento y Test de Calidad Definitivo

```

library(terra)

# — CONFIGURACIÓN DE LAS RUTAS DEL ESPACIO DE
TRABAJO —
directorio_master <-
"E:/TFG/2026/DATOS/CORRECCION/VARIABLES
PREDICTORAS/PROCESADAS/LIMPIAS_PARA_MAXENT"
anos_a_procesar <- 2022:2025

# Definición del bloque de variables a
armonizar (BIO12 a BIO19)
bloque_precipitacion <- paste0("BIO", 12:19)

```

```

# MAESTRO GEOMÉTRICO DE REFERENCIA (Capa
histórica validada en ETRS89 / UTM Zona 30N)
# Se utiliza este archivo como molde único para
garantizar la perfecta alineación de la malla
ruta_molde_referencia <-
"E:/TFG/2026/DATOS/CORRECCION/VARIABLES
PREDICTORAS/PROCESADAS/LIMPIAS_PARA_MAXENT/2021
/BIO13_iberica_2021.tif"

if (!file.exists(ruta_molde_referencia)) {
stop("CRÍTICO: No se encuentra el archivo
maestro de referencia geométrica.")
}

# Carga de la estructura geométrica de destino
(EPSC:25830)
molde <- rast(ruta_molde_referencia)

# — BUCLE INTERATIVO DE ARMONIZACIÓN
GEOMÉTRICA MULTIVARIABLE —
for (ano in anos_a_procesar) {

cat("\n-----
-----\n")
cat("Armonizando geometría e interpolando
bloque temporal del año:", ano, "\n")

cat("-----
-----\n")

carpeta_anual <- file.path(directorio_master,
as.character(ano))

# Procesamiento secuencial de cada variable
del bloque seleccionado
for (variable in bloque_precipitacion) {
ruta_raster <- file.path(carpeta_anual,
paste0(variable, "_iberica_", ano, ".tif"))

if (file.exists(ruta_raster)) {
cat("Procesando predictor:", variable,
"-> ")

# Carga de la variable bioclimática bruta
en formato geográfico (WGS84)
img <- rast(ruta_raster)

# 1. Reproyección cartográfica con
interpolación bilineal al sistema métrico
img_projected <- project(img, molde,
method = "bilinear")

# 2. Recorte perimetral con el troquel de
la máscara continental
img_clean <- mask(img_projected, molde)

# 3. Guardado físico y optimización
estructural con compresión LZW
writeRaster(img_clean,
ruta_raster,
overwrite = TRUE,
datatype = "FLT4S",
gdal = "COMPRESS=LZW")

# 4. Control de calidad topológico
automatizado

```

```

    test_veredicto <- compareGeom(molde,
img_clean, stopOnError = FALSE)
    if (test_veredicto) {
      cat("CONCORDANCIA ABSOLUTA (TRUE)\n")
    } else {
      cat("ERROR DE ALINEACIÓN CRÍTICO\n")
    }
  } else {
    # Control por si alguna variable del
    bloque no fuera requerida en el directorio
    warning(paste("Archivo no localizado
para:", variable, "en el año:", ano))
  }
}
}
cat("\n=== PROCESO DE ARMONIZACIÓN TOTALMENTE
COMPLETADO ===\n")

```

Anexo VIII: Script en R para la Consolidación Temporal, Muestreo Espacial, Selección por VIF Paso a Paso y Reproyección de Presencias

```

=====
=====
# SCRIPT: Consolidación, Filtrado VIF y
Preparación para Biomod2
# Península Ibérica – TFG 2026
#
=====
=====

# — 0. Configuración del Entorno y Corrección
PROJ -----
Sys.setenv(
  PROJ_LIB = system.file("proj", package =
"terra"),
  GDAL_DATA = system.file("gdal", package =
"terra")
)

# Librerías necesarias para el flujo de trabajo
paquetes_necesarios <- c("terra", "corrplot",
"sf")
paquetes_a_instalar <-
paquetes_necesarios[!paquetes_necesarios %in%
installed.packages()[, "Package"]]

if (length(paquetes_a_instalar) > 0) {
  install.packages(paquetes_a_instalar,
dependencies = TRUE)
}
invisible(lapply(paquetes_necesarios, library,
character.only = TRUE))

# — 1. Definición de Rutas de Trabajo
-----

RUTA_STACK <-
r"(E:\TFG\2026\DATOS\CORRECCION\VARIABLES
PREDICTORAS\PROCESADAS\stack_variables-PI.tif)"

```

```

RUTA_PUNTOS <-
r"(E:\TFG\2026\DATOS\CORRECCION\VARIABLES
PREDICTORAS\PRESENCIAS\presencias_originales.sh
p)"
RUTA_SALIDA <-
r"(E:\TFG\2026\DATOS\CORRECCION\VARIABLES
PREDICTORAS\PROCESADAS)"

```

— 2. Carga del Stack Multibanda Original (2001-2025) -----

```

message("— Cargando stack original de
variables...")
stack_gigante <- terra::rast(RUTA_STACK)
nombres_capas <- names(stack_gigante)
message("Stack cargado con éxito: ",
length(nombres_capas), " capas detectadas.")

```

— 3. Reducción Temporal: Cálculo de Medias Históricas -----

```

message("\n— Iniciando consolidación temporal
(Promedios 2001-2025)...")

```

```

# Identificación de variables base omitiendo el
sufijo cronológico interanual
prefijos <- unique(gsub("_[0-9]{4}$", "",
nombres_capas))
stack_promedios <- terra::rast()

```

```

for (p in prefijos) {
  indices_capas <- grep(paste0("^", p,
"_[0-9]{4}$"), nombres_capas)

```

```

  if (length(indices_capas) == 0) {
    indices_capas <- grep(p, nombres_capas)
  }

```

```

  if (length(indices_capas) > 0) {
    capas_variable <-
stack_gigante[[indices_capas]]
    media_historica <-
terra::mean(capas_variable, na.rm = TRUE)

```

```

  # Estandarización y homogeneización formal
de la nomenclatura de salida
  partes <- unlist(strsplit(p, "_"))
  if (partes[1] == "BIO" && length(partes) >=
2) {

```

```

    nombre_final <- partes[2]
  } else {
    nombre_final <- partes[1]
  }

```

```

  names(media_historica) <- nombre_final
  stack_promedios <- c(stack_promedios,
media_historica)
}
}

```

```

message("Reducción temporal completada. Stack
sintetizado a ", terra::nlyr(stack_promedios),
" variables base.")

```

— 4. Muestreo Espacial y Matriz de Correlación de Pearson -----

```

message("\n— Extrayendo muestra espacial de
10,000 píxeles...")
set.seed(42) # Garantizar la replicabilidad
exacta del muestreo aleatorio

```

```

muestra_raw <-
terra::spatSample(stack_promedios, size =
10000, method = "random", na.rm = TRUE)
muestra_df <- as.data.frame(muestra_raw)
muestra_limpia <- na.omit(muestra_df)
muestra_limpia <-
as.data.frame(data.matrix(muestra_limpia))

message("— Calculando matriz de correlación de
Pearson...")
cor_matrix <- cor(muestra_limpia, method =
"pearson", use = "complete.obs")

# Exportación del gráfico analítico de
correlación lineal
png(file.path(RUTA_SALIDA,
"01_matriz_correlacion.png"), width = 1200,
height = 1200, res = 120)
corrplot::corrplot(cor_matrix, method =
"color", type = "upper", tl.cex = 0.7,
tl.col = "black",
addCoef.col = "black", number.cex = 0.5,
title = "Matriz de
correlación de Pearson (Variables Promedio)",
mar = c(0, 0, 2, 0))
dev.off()

# — 5. Análisis VIF (Bucle Algorítmico
Stepwise)

message("\n— Iniciando cálculo iterativo del
Factor de Inflación de la Varianza (VIF)...")

df_vif <- muestra_limpia
umbral_vif <- 3 # Criterio estricto de
exclusión para mitigar el sobreajuste muestral
keep_going <- TRUE

while (keep_going) {
  var_nombres <- colnames(df_vif)
  if (length(var_nombres) <= 1) {
    keep_going <- FALSE
    break
  }

  # Estimación matemática del VIF mediante
modelos lineales por pares
  vif_valores <- sapply(var_nombres,
function(x) {
    form <- as.formula(paste(x, "~ ."))
    model <- lm(form, data = df_vif)
    r2 <- summary(model)$r.squared
    1 / (1 - r2)
  })

  max_vif <- max(vif_valores, na.rm = TRUE)

  # Exclusión iterativa de la variable con
mayor tasa de inflación de varianza
  if (max_vif > umbral_vif) {
    peor_var <-
names(vif_valores)[which.max(vif_valores)]
    message(" [Eliminada por colinealidad] ->
", peor_var, " (VIF = ", round(max_vif, 2),
")")
    df_vif <- df_vif[, !colnames(df_vif) %in%
peor_var, drop = FALSE]

```

```

} else {
  keep_going <- FALSE
}
}

variables_seleccionadas <- colnames(df_vif)
message("\nVariables ecológicas finales
seleccionadas (VIF < 3):")
print(variables_seleccionadas)

# — 6. Reproyección Cartográfica Automatizada
de Presencias —————
message("\n— Iniciando armonización
cartográfica de registros bióticos...")
if (file.exists(RUTA_PUNTOS)) {
  puntos_presencia <- sf::st_read(RUTA_PUNTOS,
quiet = TRUE)

  # Extracción del sistema de referencia
espacial nativo de las variables (EPSG:4326)
  crs_destino <- terra::crs(stack_promedios)

  # Reproyección formal de los registros
bióticos para resolver el spatial mismatch
  puntos_reproyectados <-
sf::st_transform(puntos_presencia, crs = 4326)

  # Exportación del archivo vectorial final
armonizado
  sf::st_write(puntos_reproyectados,
file.path(RUTA_SALIDA,
"presencias_reproyectadas_4326.shp"),
delete_dsn = TRUE, quiet = TRUE)
  message("Registros bióticos reproyectados con
éxito a EPSG:4326.")
} else {
  message("Aviso: No se localizó el archivo de
presencias en la ruta provista.")
}

# — 7. Exportar Stack Reducido y Tablas de
Control —————
message("\n— Generando archivos ráster de
salida para Biomod2...")
stack_final <-
stack_promedios[[variables_seleccionadas]]

# Almacenamiento del stack multibanda
definitivo bajo compresión geoespacial LZW
ruta_stack_final <- file.path(RUTA_SALIDA,
"stack_variables_VIF.tif")
terra::writeRaster(stack_final,
ruta_stack_final, overwrite = TRUE, gdal =
c("COMPRESS=LZW"))

# Compilación y generación de la tabla de
reporte analítico
nombres_totales_limpios <-
names(stack_promedios)
tabla_vars <- data.frame(variable =
variables_seleccionadas, seleccionada = "Sí
(Apta para Biomod2)")
tabla_excluidas <- data.frame(
variable =
nombres_totales_limpios[!nombres_totales_limpio
s %in% variables_seleccionadas],

```



```

seleccionada = "NO – Excluida por alta
colinealidad (VIF)"
)
tabla_final <- rbind(tabla_vars,
tabla_excluidas)
write.csv(tabla_final, file.path(RUTA_SALIDA,
"02_variables_VIF_seleccion.csv"), row.names =
FALSE)

message("\n=====
=====")
message(" Proceso completado con éxito.
Variables y registros listos para Biomod2.")
message("=====
=====")

```

Anexo IX: Script en R para el ajuste de modelos de distribución de especies (SDMs), ensamblado (ensemble modelling), proyección temporal (2001–2024) y análisis de tendencias de Mann-Kendall / Sen's slope

```

library(biomod2)
library(terra)
library(trend)
library(dplyr)

# =====
# 1. CONFIGURACIÓN Y PREPARACIÓN DE DATOS
# =====
# Carga de presencias de la especie (columnas:
x, y, pres)
presencias <-
read.csv("E:/TFG/2026/datos/presencias_especie.
csv")
coords <- presencias[, c("x", "y")]
resp_var <- presencias$pres

# Carga de predictores ambientales de
referencia
predictores <-
rast("E:/TFG/2026/capas/predictores_2001.tif")

# Estrategia de Pseudo-Ausencias (PA):
# Use "disk" (5 a 200 km) para especies
restringidas / endémicas
# Use "random" para especies generalistas
PA_strategy <- "disk"

myBiomodData <- BIOMOD_FormatingData(
  resp.var = resp_var,
  expl.var = predictores,
  resp.xy = coords,
  resp.name = "Especie_Estudio",
  PA.nb.rep = 3,
  PA.nb.absences = 10000,
  PA.strategy = PA_strategy,
  PA.dist.min = 5000, # 5 km
  PA.dist.max = 200000 # 200 km
)

```

```

# =====
# 2. MODELIZACIÓN INDIVIDUAL (BIOMOD_Modeling)
# =====
myBiomodModelOut <- BIOMOD_Modeling(
  bm.format = myBiomodData,
  models = c("GLM", "GAM", "RF", "GBM", "ANN",
"MAXNET"),
  CV.strategy = "random",
  CV.nb.rep = 10,
  CV.perc = 0.8,
  metric.eval = c("TSS", "ROC"),
  var.import = 3,
  nb.cpu = 4
)

# =====
# 3. MODELO DE CONJUNTO
(BIOMOD_EnsembleModeling)
# =====
myBiomodEM <- BIOMOD_EnsembleModeling(
  bm.mod = myBiomodModelOut,
  models.eval.meth = c("TSS"),
  em.by = "all",
  metric.select = "TSS",
  metric.select.thresh = 0.6,
  em.algo = c("EMwmean") # Media ponderada por
TSS
)

# =====
# 4. PROYECCIÓN ANUAL (2001-2024) Y GENERACIÓN
DE HSI
# =====
hsi_lista <- list()

for (year in 2001:2024) {
  # Cargar variables dinámicas del año en curso
  env_year <-
rast(paste0("E:/TFG/2026/capas/predictores_",
year, ".tif"))

  # Proyección Ensemble
myBiomodEF <- BIOMOD_EnsembleForecasting(
  bm.em = myBiomodEM,
  proj.name = paste0("HSI_", year),
  new.env = env_year,
  models.chosen = "all"
)

  # Extraer mapa HSI y escalar a rango [0, 1]
r_hsi <- get_predictions(myBiomodEF)[[1]] /
1000
hsi_lista[[as.character(year)]] <- r_hsi
}

# Crear serie temporal ráster de 24 capas
hsi_stack <- rast(hsi_lista)

# =====
# 5. ANÁLISIS DE TENDENCIAS (MANN-KENDALL Y
SEN'S SLOPE)
# =====
# Función celda a celda para calcular la
pendiente de Sen y p-valor de Mann-Kendall
fun_sen_mk <- function(v) {

```

```

    if (any(is.na(v))) return(c(slope = NA,
pvalue = NA))
    res <- trend::sens.slope(ts(v, start = 2001,
end = 2024))
    return(c(slope = res$estimates[["Sen's slope
"]], pvalue = res$p.value))
}

# Aplicar la función sobre toda la superficie
del mapa
mapa_tendencia <- app(hsi_stack, fun =
fun_sen_mk, cores = 4)
names(mapa_tendencia) <- c("Sen_Slope",
"MK_pvalue")

# Filtrar y aislar píxeles con tendencia
significativa (p < 0,05)
mapa_slope_sig <- mapa_tendencia[["Sen_Slope"]]
mapa_slope_sig[mapa_tendencia[["MK_pvalue"]] >=
0.05] <- NA

# Exportar resultados en formato TIFF
writeRaster(mapa_tendencia[["Sen_Slope"]],
"E:/TFG/2026/outputs/Sen_Slope_Completo.tif",
overwrite = TRUE)
writeRaster(mapa_slope_sig,
"E:/TFG/2026/outputs/Sen_Slope_Significativo_p0
05.tif", overwrite = TRUE)

```

Anexo X: CÓDIGO R PARA EL PROCESAMIENTO ESPACIAL Y ANÁLISIS ESTADÍSTICO # EVALUACIÓN DE TENDENCIAS EN LA RED NATURA 2000 (2001-2024)

```

#
-----
----- # 1. Carga de
librerías y preparación de datos espaciales #
-----
----- library(terra)
library(sf) library(dplyr) library(rstatix) #
Carga de la delimitación de la Red Natura 2000
y ráster de referencia (1x1 km) rn2000_vector
<-
st_read("datos/red_natura_2000_peninsula.gpkg")
raster_ref <-
rast("datos/raster_referencia_1km.tif") #
Reproyección a EPSG:4326 (WGS84) y
rasterización rn2000_4326 <-
st_transform(rn2000_vector, crs = 4326)
rn2000_mask <- rasterize(rn2000_4326,
raster_ref, field = 1, background = 0)
names(rn2000_mask) <- "RN2000" #
-----
----- # 2. Función
para Estadística Descriptiva y Umbrales de
Ganancia/Pérdida (Tabla 1) #
-----
----- UMBRAL_GANANCIA
<- 0.00001 UMBRAL_PERDIDA <- -0.00001
calcular_tabla_descriptiva <-
function(raster_slope, mascara_rn, nombre_capa)

```

```

{ # Extraer datos ráster en data.frame
suprimiendo valores NAs df <- data.frame( slope
= values(raster_slope, mat = FALSE), rn2000 =
values(mascara_rn, mat = FALSE) ) %>% na.omit()
df <- df %>% mutate(zona = ifelse(rn2000 == 1,
"Dentro_RN2000", "Fuera_RN2000")) # Cálculo de
métricas resumen <- df %>% group_by(zona) %>%
summarise( Capa = nombre_capa, Total_Pixeles =
n(), Slope_Medio = mean(slope), Slope_Mediana =
median(slope), Px_Ganancia = sum(slope >
UMBRAL_GANANCIA), Px_Perdida = sum(slope <
UMBRAL_PERDIDA), Pct_Ganancia =
round((Px_Ganancia / Total_Pixeles) * 100, 2),
Pct_Perdida = round((Px_Perdida /
Total_Pixeles) * 100, 2), .groups = "drop" )
return(resumen) } #
-----
----- # 3. Función
para Test U de Mann-Whitney y Tamaño del Efecto
r (Tabla 2) #
-----
-----
calcular_test_y_efecto <-
function(raster_slope, mascara_rn, nombre_capa)
{ df <- data.frame( slope =
values(raster_slope, mat = FALSE), rn2000 =
values(mascara_rn, mat = FALSE) ) %>% na.omit()
%>% mutate(zona = factor(ifelse(rn2000 == 1,
"Dentro_RN2000", "Fuera_RN2000"))) # Medianas
por dominio espacial mediana_dentro <-
median(df$slope[df$zona == "Dentro_RN2000"])
mediana_fuera <- median(df$slope[df$zona ==
"Fuera_RN2000"]) # Prueba U de Mann-Whitney
(muestras independientes, asunción no
paramétrica) test_mw <- wilcox.test(slope ~
zona, data = df, exact = FALSE) # Tamaño del
efecto r de Rosenthal: r = |Z| / sqrt(N)
efecto_r <- df %>% wilcox_effsize(slope ~ zona,
ci = FALSE) val_r <- round(efecto_r$effsize, 3)
# Clasificación cualitativa del efecto
relevancia <- case_when( val_r < 0.02 ~
"Despreciable (Sin efecto diferencial)", val_r
< 0.10 ~ "Pequeña", val_r >= 0.10 ~ "Moderada"
) resultado <- data.frame( Capa_grupo =
nombre_capa, Mediana_Dentro =
round(mediana_dentro, 6), Mediana_Fuera =
round(mediana_fuera, 6), p_valor =
ifelse(test_mw$p.value < 0.001, "< 0,001",
round(test_mw$p.value, 4)), Tamano_Efecto_r =
val_r, Relevancia_Ecologica = relevancia )
return(resultado) } #
-----
----- # 4. Bucle de
ejecución para capas de Grupos y Amenaza #
-----
----- # Lista de
rásters de Sen's slope previamente procesados
lista_rasters <-
list.files("datos/sens_slope/", pattern =
"\\.tif$", full.names = TRUE) resultados_tabla1
<- list() resultados_tabla2 <- list() for
(archivo in lista_rasters) { nombre_capa <-
gsub("\\.tif$", "", basename(archivo)) r_slope
<- rast(archivo) # Generar registros para Tabla
1 y Tabla 2 resultados_tabla1[[nombre_capa]] <-
calcular_tabla_descriptiva(r_slope,
rn2000_mask, nombre_capa)

```

```

resultados_tabla2[[nombre_capa]] <-
calcular_test_y_efecto(r_slope, rn2000_mask,
nombre_capa) } # Consolidación final de tablas
df_tabla1_final <- bind_rows(resultados_tabla1)
df_tabla2_final <- bind_rows(resultados_tabla2)
# Exportación de resultados a CSV para
maquetación write.csv(df_tabla1_final,
"resultados/Tabla1_Descriptivos_RN2000.csv",
row.names = FALSE) write.csv(df_tabla2_final,
"resultados/Tabla2_MannWhitney_EfectoR.csv",
row.names = FALSE)

```

Anexo XI: Script en R para el análisis inferencial de las tendencias de idoneidad del hábitat (pendiente de Sen) por grupos taxonómicos y categorías de amenaza de la UICN

```

library(terra)
library(dplyr)
library(rstatix)

# Definición de rutas directas
dir_taxa <-
"E:/TFG/2026/outputs_summary2/mapasPromedio/Gru
poTaxonomico"
dir_uicn <-
"E:/TFG/2026/outputs_summary2/mapasPromedio/Cla
sificacionUICN"

# =====
# 1. GRUPOS TAXONÓMICOS
# =====
df_taxa <- bind_rows(
  data.frame(slope =
values(rast(file.path(dir_taxa,
"Mapa_Promedio_Grupo_anfibios.tif")), mat =
FALSE), grupo = "anfibios"),
  data.frame(slope =
values(rast(file.path(dir_taxa,
"Mapa_Promedio_Grupo_aves.tif")), mat = FALSE),
grupo = "aves"),
  data.frame(slope =
values(rast(file.path(dir_taxa,
"Mapa_Promedio_Grupo_mamiferos.tif")), mat =
FALSE), grupo = "mamiferos"),
  data.frame(slope =
values(rast(file.path(dir_taxa,
"Mapa_Promedio_Grupo_reptiles.tif")), mat =
FALSE), grupo = "reptiles"),
  data.frame(slope =
values(rast(file.path(dir_taxa,
"Mapa_Promedio_Grupo_vasculares.tif")), mat =
FALSE), grupo = "vasculares")
) %>% na.omit()

# Muestreo aleatorio representativo para evitar
colapsar la RAM durante el post-hoc
set.seed(123)
df_taxa_sample <- df_taxa %>%
sample_n(min(50000, nrow(df_taxa)))

```

```

# Pruebas estadísticas
kw_taxa <- kruskal.test(slope ~ grupo, data =
df_taxa_sample)
dunn_taxa <- df_taxa_sample %>% dunn_test(slope
~ grupo, p.adjust.method = "bonferroni")

```

```

# =====
# 2. CATEGORÍAS UICN
# =====
df_uicn <- bind_rows(
  data.frame(slope =
values(rast(file.path(dir_uicn,
"Mapa_Promedio_Amenaza_EN.tif")), mat = FALSE),
amenaza = "EN"),
  data.frame(slope =
values(rast(file.path(dir_uicn,
"Mapa_Promedio_Amenaza_LC.tif")), mat = FALSE),
amenaza = "LC"),
  data.frame(slope =
values(rast(file.path(dir_uicn,
"Mapa_Promedio_Amenaza_NT.tif")), mat = FALSE),
amenaza = "NT"),
  data.frame(slope =
values(rast(file.path(dir_uicn,
"Mapa_Promedio_Amenaza_VU.tif")), mat = FALSE),
amenaza = "VU")
) %>% na.omit()

```

```

df_uicn_sample <- df_uicn %>%
sample_n(min(50000, nrow(df_uicn)))

# Pruebas estadísticas
kw_uicn <- kruskal.test(slope ~ amenaza, data =
df_uicn_sample)
dunn_uicn <- df_uicn_sample %>% dunn_test(slope
~ amenaza, p.adjust.method = "bonferroni")

```

```

# =====
# RESULTADOS
# =====
cat("\n--- RESULTADOS GRUPOS TAXONÓMICOS
---\n")
print(kw_taxa)
print(dunn_taxa, n = 20)

cat("\n--- RESULTADOS CATEGORÍAS UICN ---\n")
print(kw_uicn)
print(dunn_uicn, n = 20)

```

Anexo XII: Script en R para la evaluación espacial de la tendencia de idoneidad del hábitat y generación gráfica dentro y fuera de la Red Natura 2000

```

library(terra)
library(dplyr)
library(tidyr)
library(ggplot2)

# 1. Carga de datos

```

```

dir_taxa <-
"E:/TFG/2026/outputs_summary2/mapasPromedio/Gru
poTaxonomico"
dir_uicn <-
"E:/TFG/2026/outputs_summary2/mapasPromedio/Cla
sificacionUICN"
rn2000_mask <-
rast("E:/TFG/2026/capas/RedNatura2000_mask.tif"
)

capas <- c(
  list.files(dir_taxa, pattern = "\\..tif$",
full.names = TRUE),
  list.files(dir_uicn, pattern = "\\..tif$",
full.names = TRUE)
)

# 2. Función de procesamiento zonal optimizada
obtener_metricas_zona <- function(ruta_raster,
mascara) {
  r <- rast(ruta_raster)
  nombre <-
tools::file_path_sans_ext(basename(ruta_raster))
)

# Reproyectar/alinear máscara si las
geometrías no coinciden
if (!compareGeom(r, mascara, stopOnError =
FALSE)) {
  mascara <- resample(mascara, r, method =
"near")
}

# Clasificación de tendencia: 1 = Ganancia
(>0), -1 = Pérdida (<0), 0 = Sin cambio (0)
r_clasificado <- classify(r, rmatrix =
matrix(c(-Inf, 0, -1, 0, 0, 0, 0, Inf, 1),
ncol = 3, byrow = TRUE))

# Conteo de píxeles por zona utilizando
crosstab de terra
tabla_conteo <- crosstab(c(mascara,
r_clasificado), long = TRUE)
colnames(tabla_conteo) <- c("rn2000",
"tendencia", "pixeles")

# Estadísticos descriptivos por zona
medial_mean <- zonal(r, mascara, fun =
"mean", na.rm = TRUE)
medial_median <- zonal(r, mascara, fun =
"median", na.rm = TRUE)

# Consolidación de resultados
resumen <- tabla_conteo %>%
  filter(!is.na(rn2000), !is.na(tendencia))
%>%
  group_by(rn2000) %>%
  summarise(
    Capa = nombre,
    Total_Pixeles = sum(pixeles),
    Px_Ganancia = sum(pixeles[tendencia ==
1]),
    Px_Perdida = sum(pixeles[tendencia ==
-1]),
    Pct_Ganancia = round((Px_Ganancia /
Total_Pixeles) * 100, 2),

```

```

    Pct_Perdida = round((Px_Perdida /
Total_Pixeles) * 100, 2),
    .groups = "drop"
  ) %>%
  mutate(
    zona = ifelse(rn2000 == 1,
"Dentro_RN2000", "Fuera_RN2000"),
    Slope_Medio = medial_mean[[2]],
    Slope_Mediana = medial_median[[2]]
  )

  return(resumen)
}

```

3. Ejecución

```

tabla_8 <- lapply(capas, obtener_metricas_zona,
mascara = rn2000_mask) %>% bind_rows()

```

Anexo XIII: Script en R para la evaluación de la significación estadística (Prueba U de Mann-Whitney) y magnitud del efecto (r de Rosenthal) entre áreas protegidas y no protegidas

```

library(terra)
library(dplyr)
library(rstatix)

```

```

# =====
# 1. RUTAS Y MÁSCARA RED NATURA 2000
# =====
dir_taxa <-
"E:/TFG/2026/outputs_summary2/mapasPromedio/Gru
poTaxonomico"
dir_uicn <-
"E:/TFG/2026/outputs_summary2/mapasPromedio/Cla
sificacionUICN"
rn2000_mask <-
rast("E:/TFG/2026/capas/RedNatura2000_mask.tif"
)

capas <- c(
  list.files(dir_taxa, pattern = "\\..tif$",
full.names = TRUE),
  list.files(dir_uicn, pattern = "\\..tif$",
full.names = TRUE)
)

# =====
# 2. FUNCIÓN DE ANÁLISIS ESTADÍSTICO (U
MANN-WHITNEY Y EFECTO r)
# =====
calcular_mann_whitney_efecto <-
function(ruta_raster, mascara) {
  r <- rast(ruta_raster)
  nombre_capa <-
tools::file_path_sans_ext(basename(ruta_raster))
)

```

```

# Resamplear si no coinciden espacialmente

```

```

if (!compareGeom(r, mascara, stopOnError =
FALSE)) {
  mascara <- resample(mascara, r, method =
"near")
}

# Unir valores de slope y máscara de
protección
df <- data.frame(
  slope = values(r, mat = FALSE),
  rn2000 = values(mascara, mat = FALSE)
) %>% na.omit()

# Etiquetar zonas
df <- df %>%
  mutate(zona = ifelse(rn2000 == 1,
"Dentro_RN2000", "Fuera_RN2000"))

# Submuestreo representativo para optimizar
rendimiento computacional
set.seed(123)
df_sample <- df %>% sample_n(min(50000,
nrow(df)))

# Medianas por zona
med_dentro <- median(df$slope[df$zona ==
"Dentro_RN2000"])
med_fuera <- median(df$slope[df$zona ==
"Fuera_RN2000"])

# Contraste de Mann-Whitney U
mw_test <- wilcox_test(slope ~ zona, data =
df_sample)

# Tamaño del efecto r de Rosenthal ( $r = |Z| / \sqrt{N}$ )
eff_size <- wilcox_effsize(slope ~ zona, data
= df_sample)
r_val <- round(eff_size$effsize, 3)

# Clasificación cualitativa de la relevancia
ecológica

relevancia <- case_when(
  r_val < 0.05 ~ "Despreciable (Sin efecto
diferencial)",
  r_val < 0.10 ~ "Pequeña",
  TRUE ~ "Moderada"
)
if (r_val >= 0.14) relevancia <- "Moderada
(Refugio claro)"

# Formato final de resultados
resultado <- data.frame(
  Capa = nombre_capa,
  Mediana_Dentro = round(med_dentro, 6),
  Mediana_Fuera = round(med_fuera, 6),
  p_valor = ifelse(mw_test$p < 0.001, "<
0,001", as.character(mw_test$p)),
  Efecto_r = r_val,
  Relevancia_Ecológica = relevancia
)

return(resultado)
}

# =====
# 3. EJECUCIÓN Y GENERACIÓN DE LA TABLA 9
# =====
tabla_9 <- lapply(capas,
calcular_mann_whitney_efecto, mascara =
rn2000_mask) %>%
  bind_rows()

print(tabla_9)
write.csv(tabla_9,
"E:/TFG/2026/outputs_summary2/Tabla_9_MannWhitn
ey_Efecto.csv", row.names = FALSE)

```

Anexo XIV: Mapas de tendencia de idoneidad de hábitat (2001-2024)

Figura A1. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Abies pinsapo*.

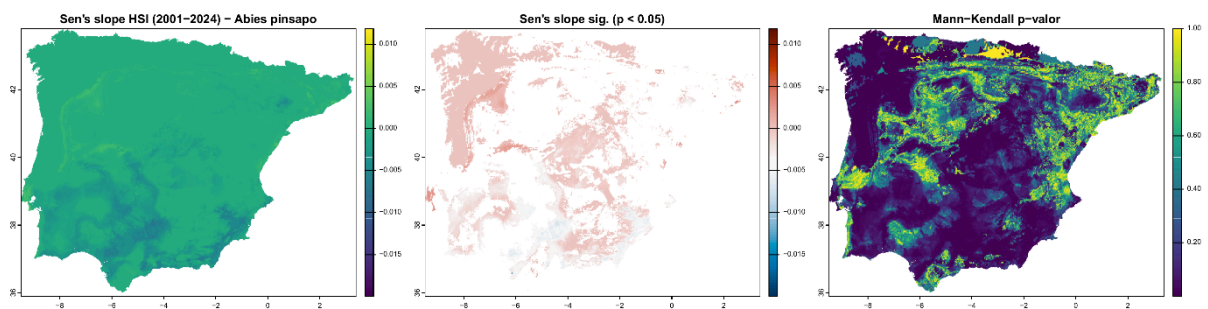


Figura A2. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Aegypius monachus*.

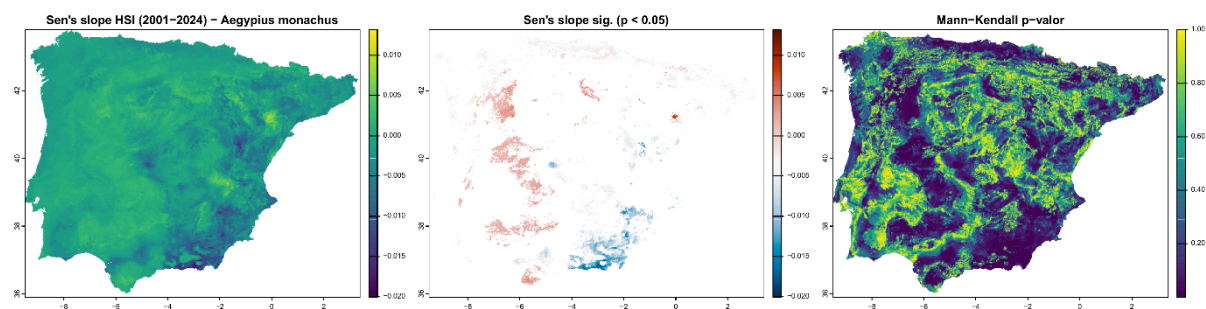


Figura A3. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Algyroides hidalgoi*.

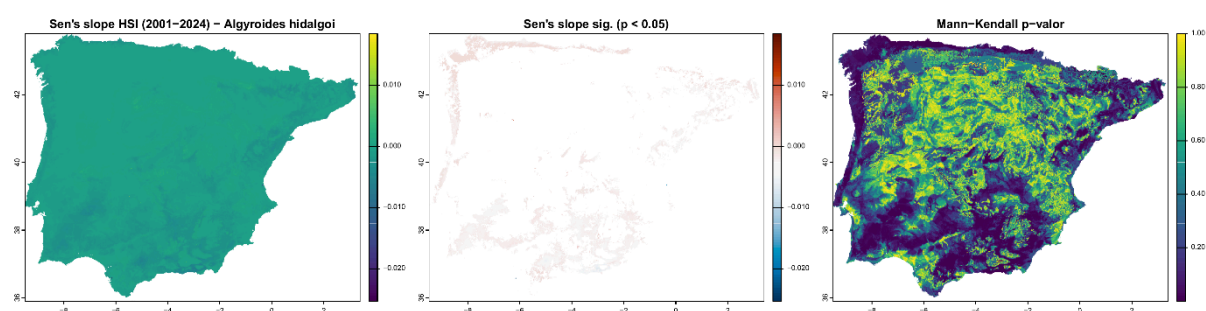


Figura A4. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Alytes cisternasii*.

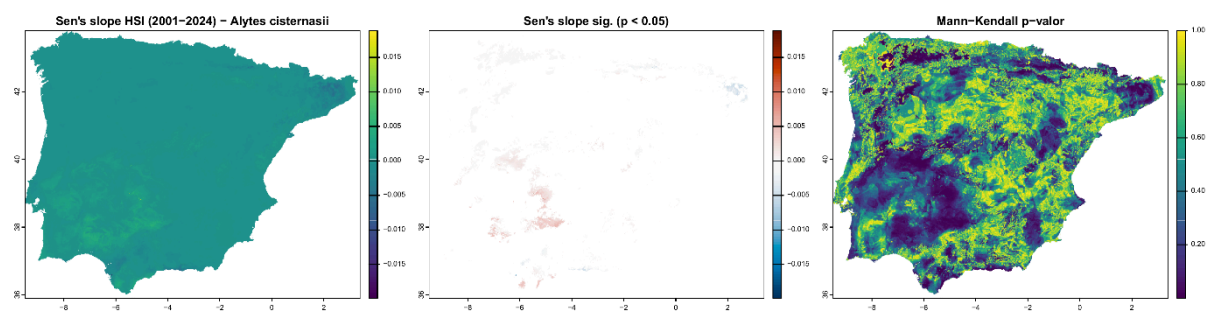


Figura A5. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Alytes dickhilleni*.

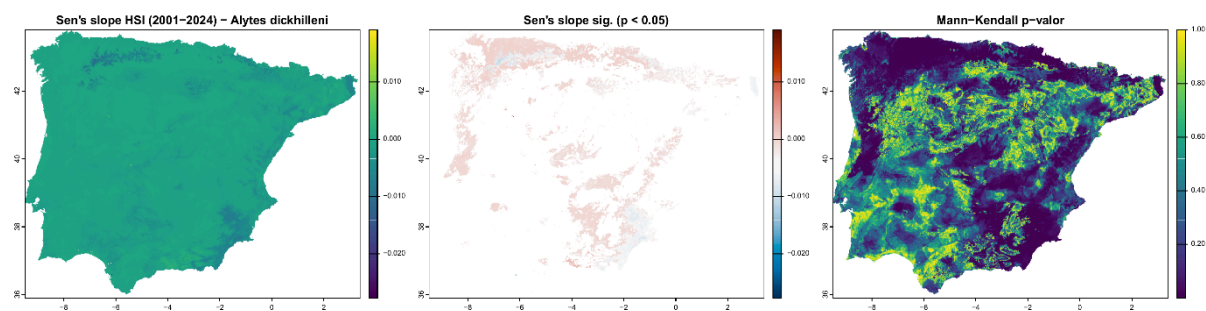


Figura A6. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Aquila adalberti*.

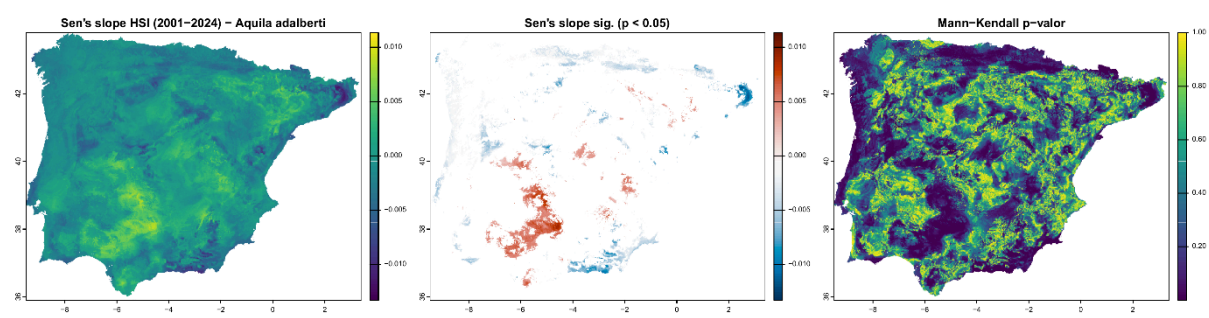


Figura A7. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Arbutus unedo*.

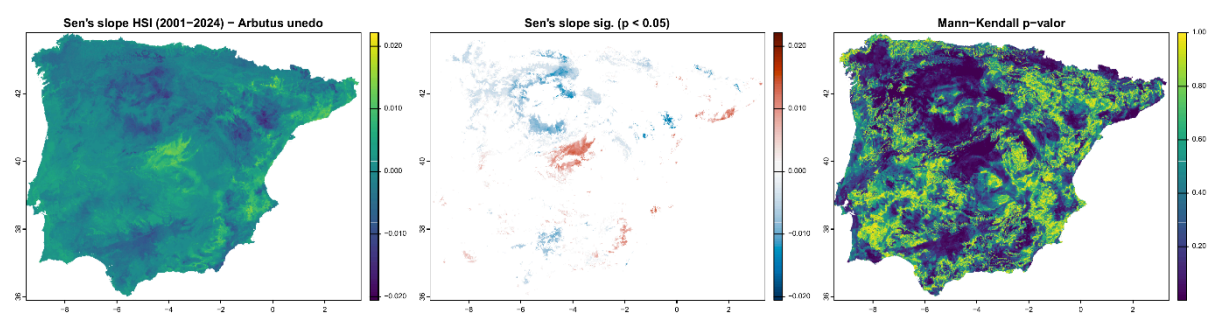


Figura A8. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Chersophilus duponti*.

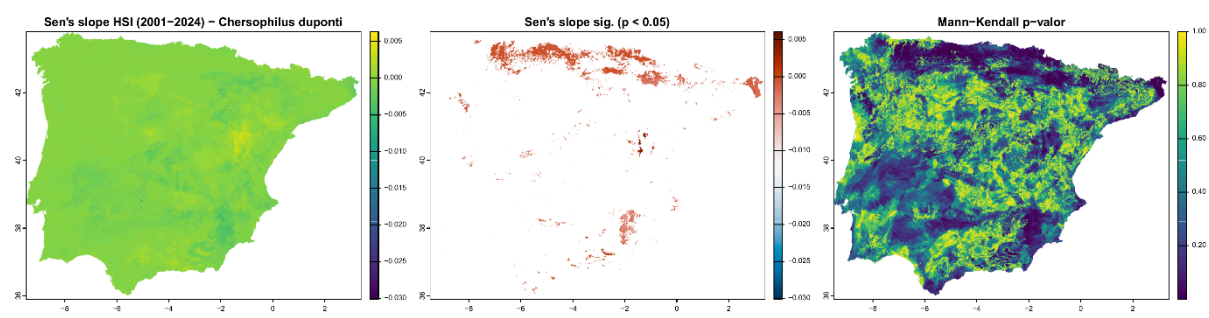


Figura A9. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Chioglossa lusitana*.

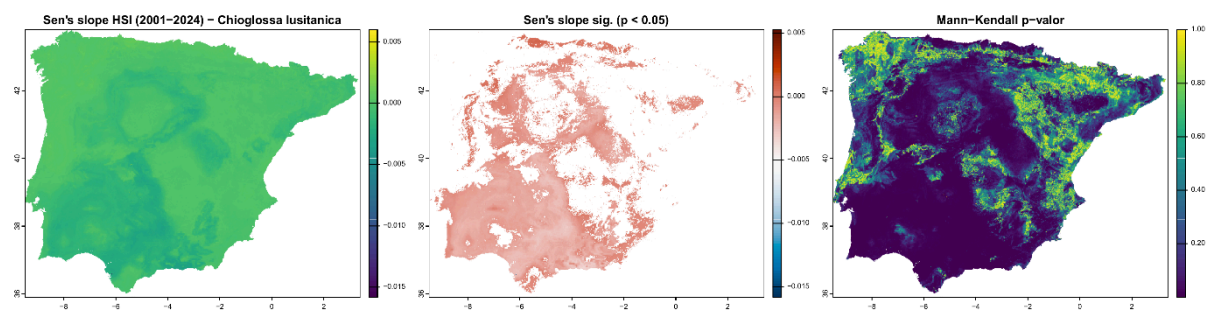


Figura A10. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Galemys pyrenaicus*.

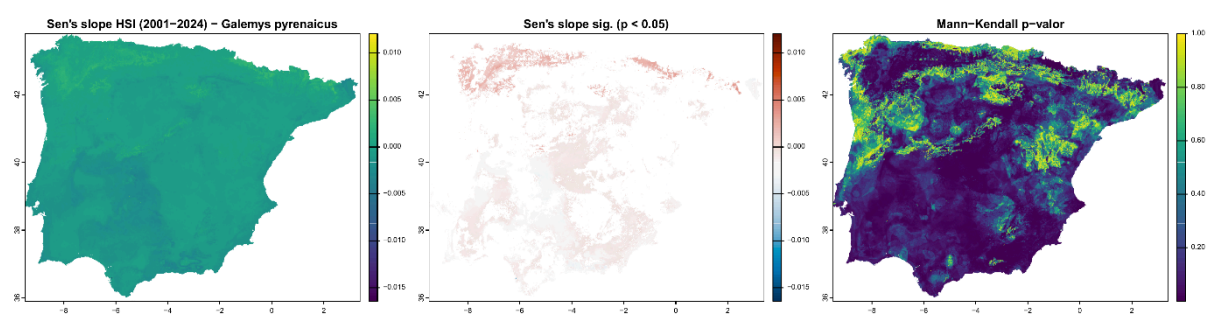


Figura A11. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Gypaetus barbatus*.

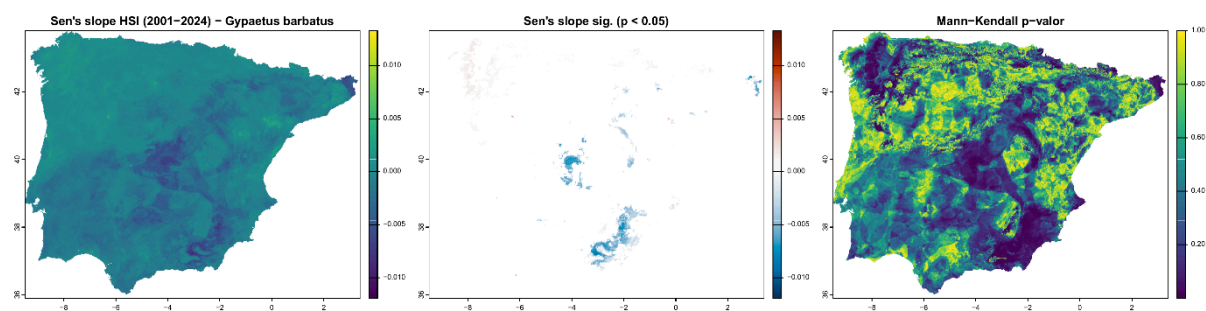


Figura A12. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Hyla meridionalis*.

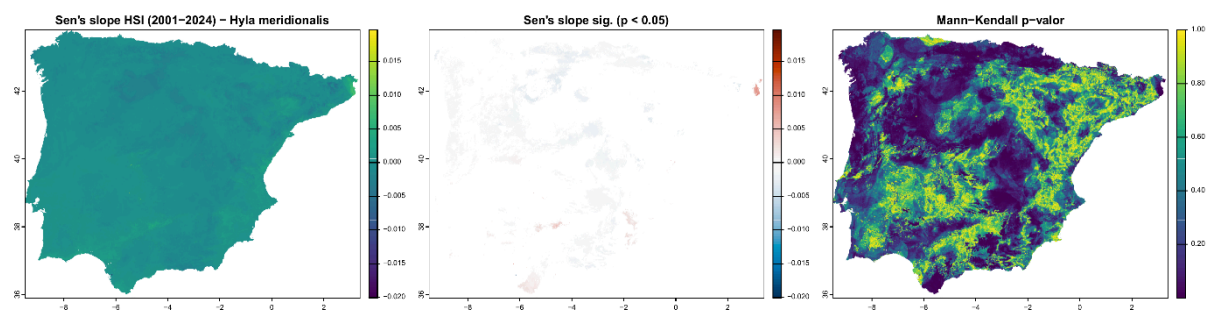


Figura A13. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Juncus heterophyllus*.

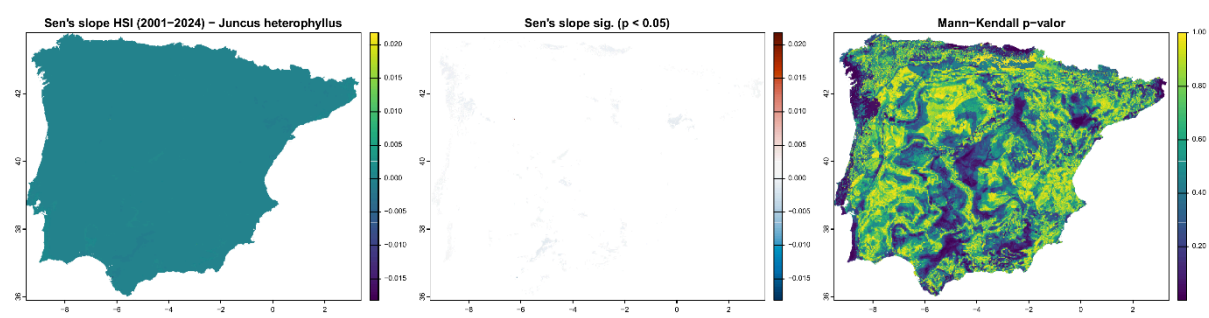


Figura A14. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Lanius senator*.

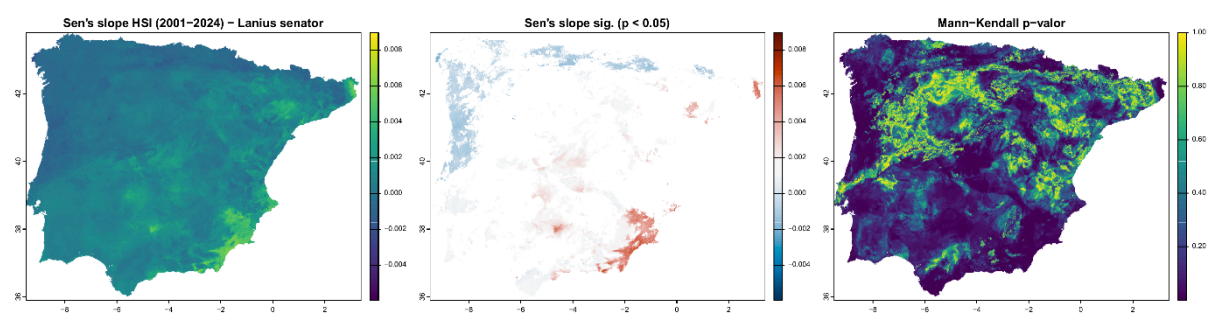


Figura A15. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Linaria ricardoi*.

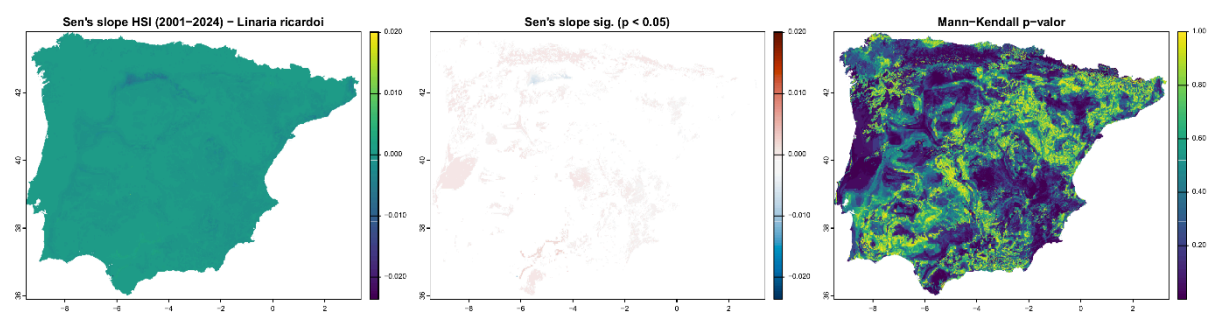


Figura A16. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Lutra lutra*.

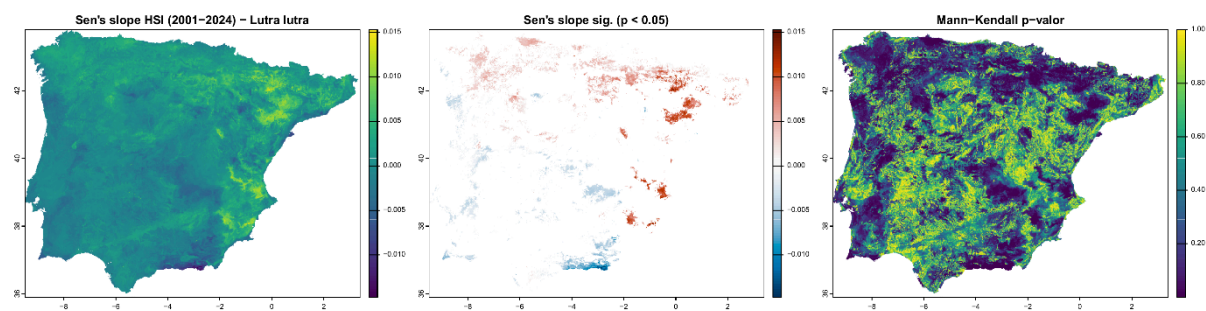


Figura A17. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Lynx pardinus*.

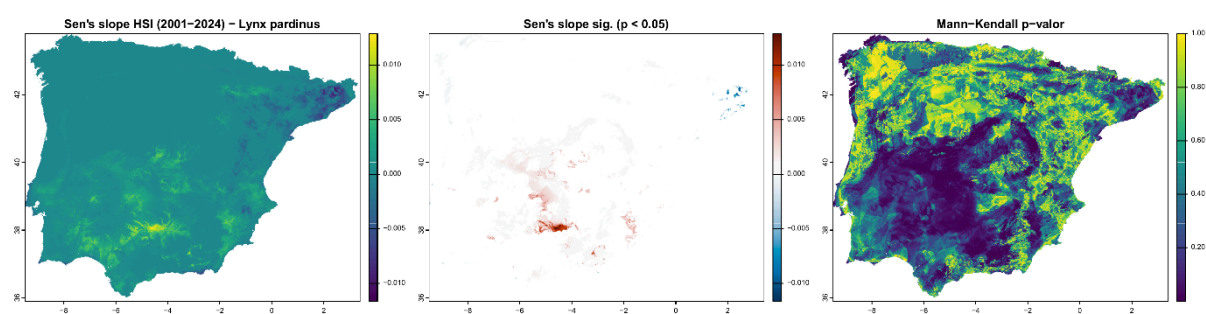


Figura A18. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Meles meles*.

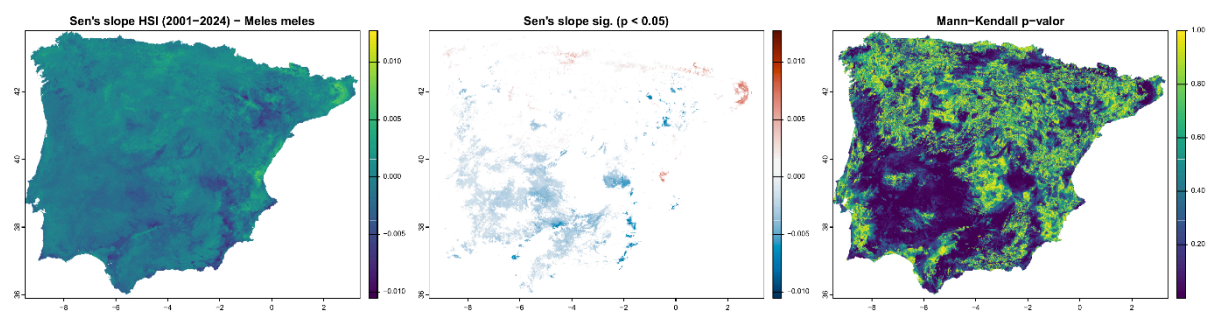


Figura A19. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Mustela lutreola*.

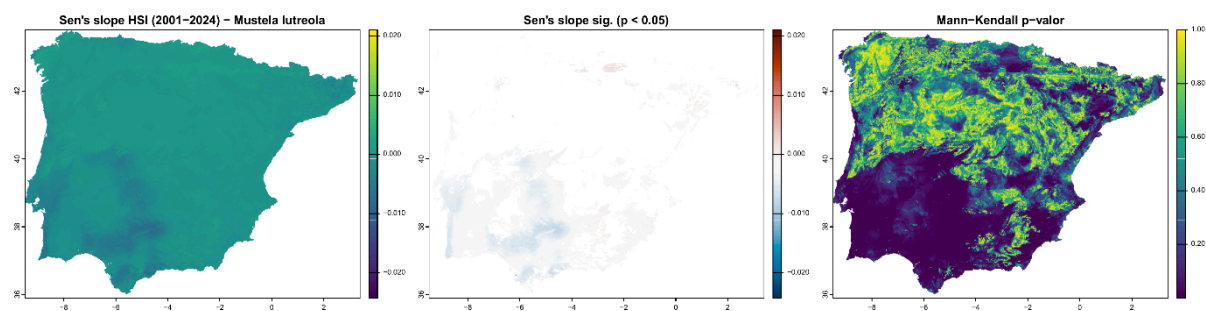


Figura A20. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Myotis capaccinii*.

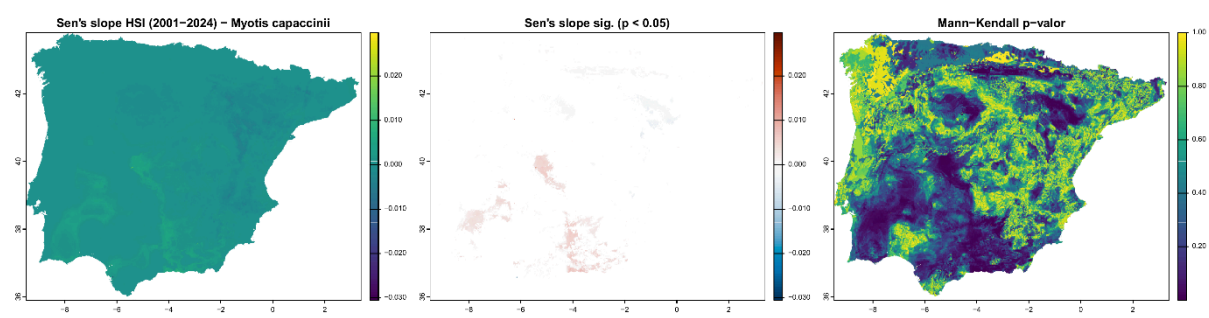


Figura A21. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Narcissus nevadensis*.

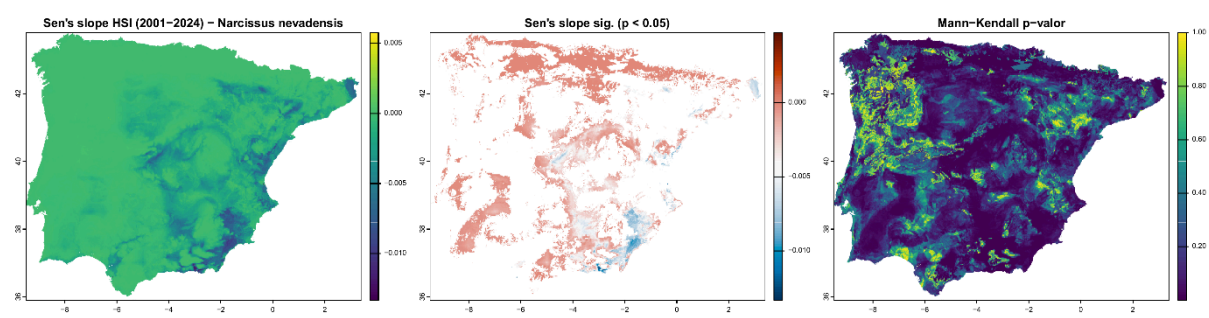


Figura A22. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Natrix maura*.

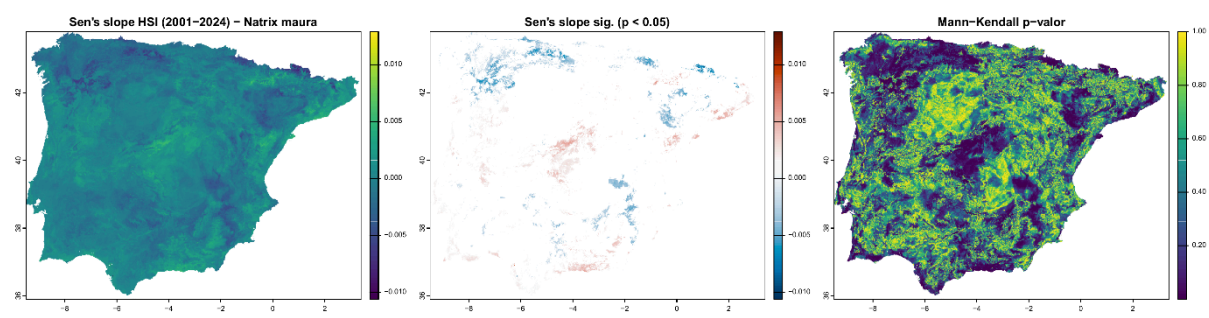


Figura A23. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Nyctalus lasiopterus*.

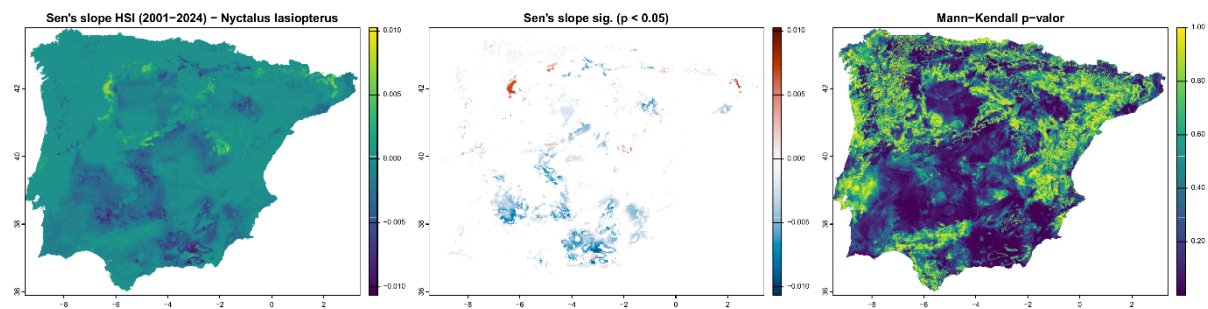


Figura A24. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Onobrychis viciifolia*.

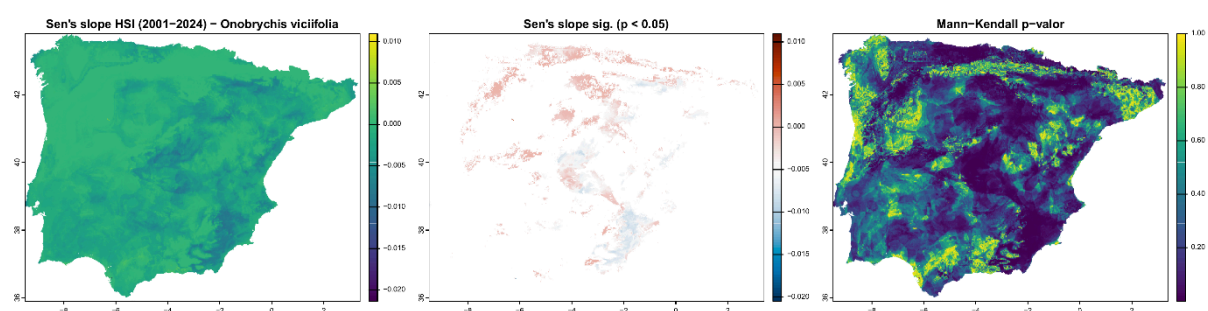


Figura A25. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Otis tarda*.

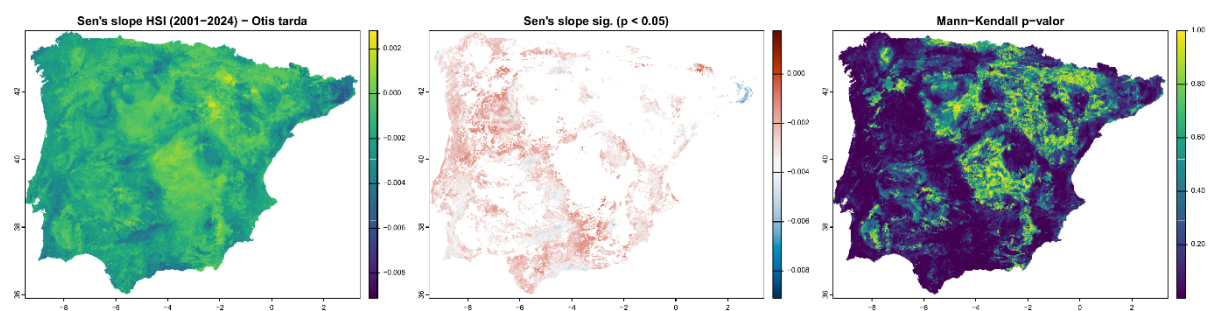


Figura A26. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Pelobates cultripes*.

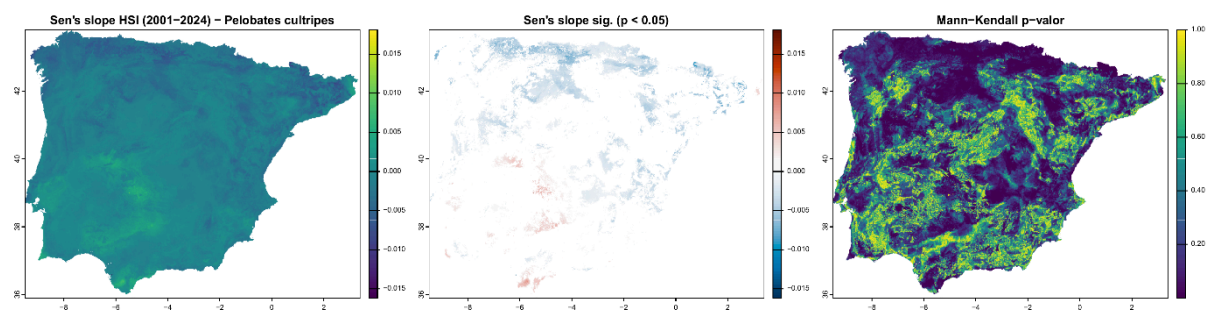


Figura A27. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Petrocoptis grandiflora*.

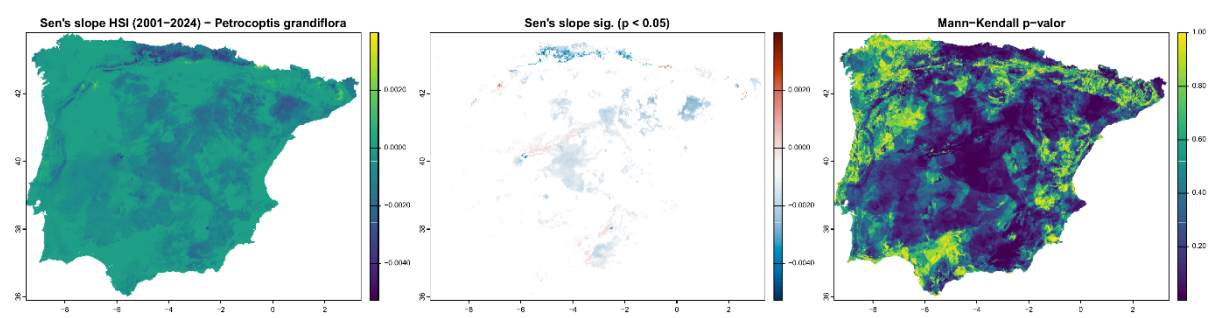


Figura A28. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Pleurodeles waltl*.

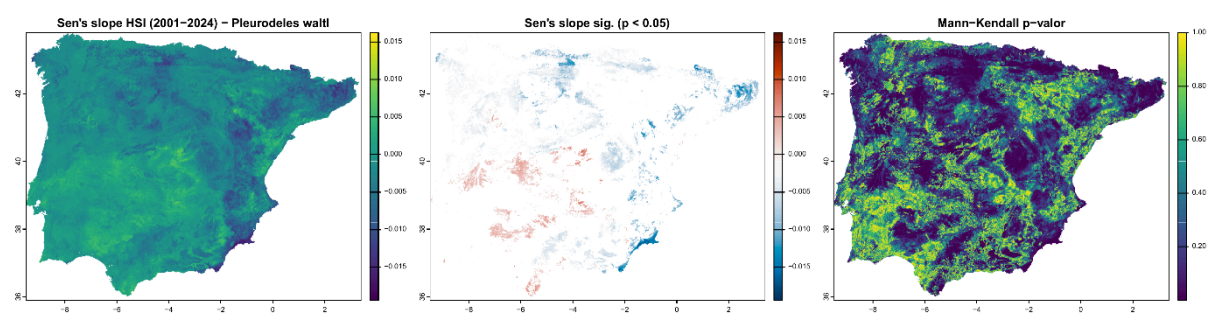


Figura A29. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Podarcis carbonelli*.

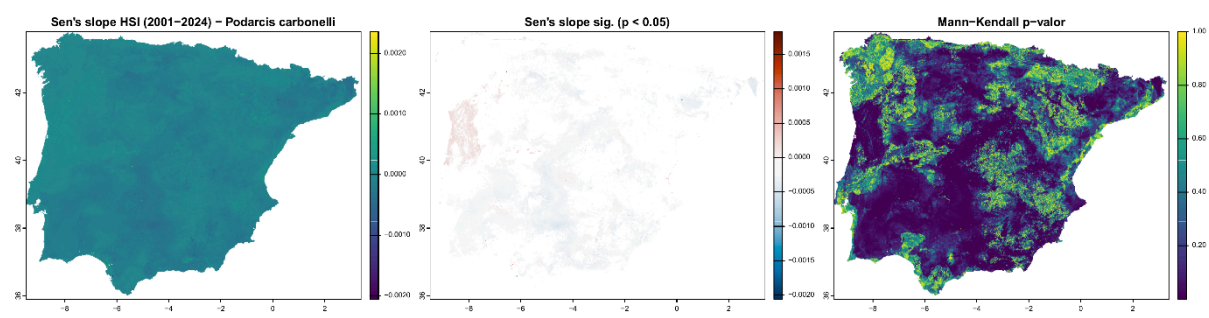


Figura A30. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Prunus lusitanica*.

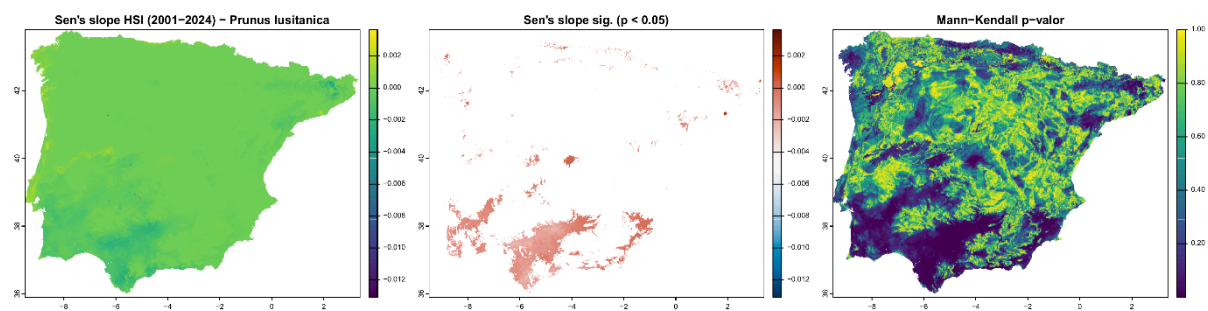


Figura A31. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Pterocles orientalis*.

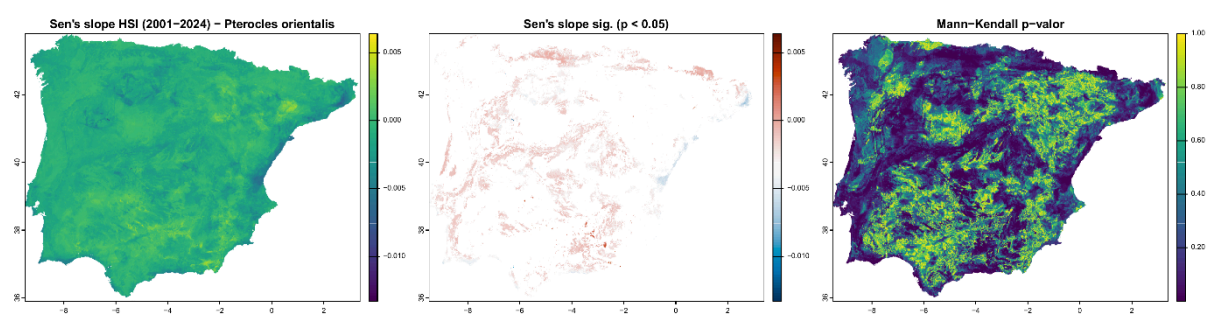


Figura A32. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Rana iberica*.

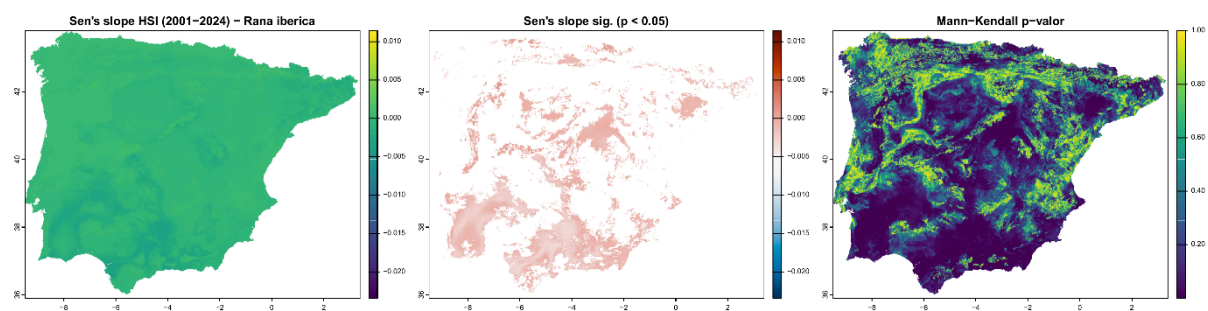


Figura A33. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Rana pyrenaica*.

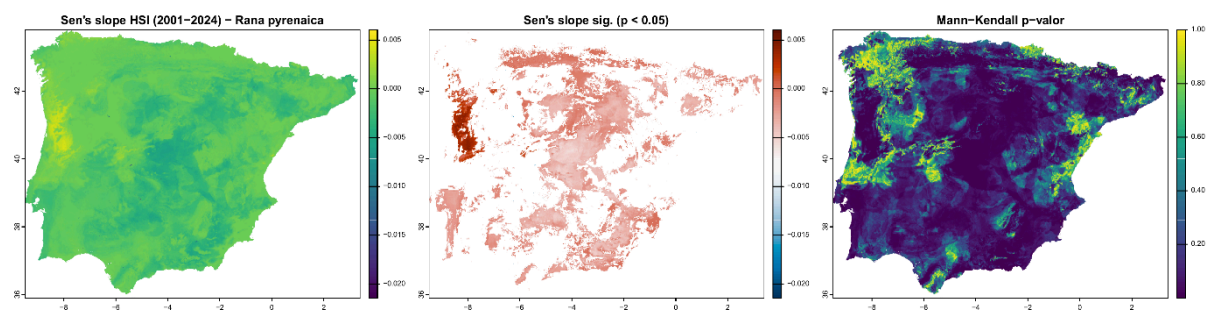


Figura A34. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Testudo graeca*.

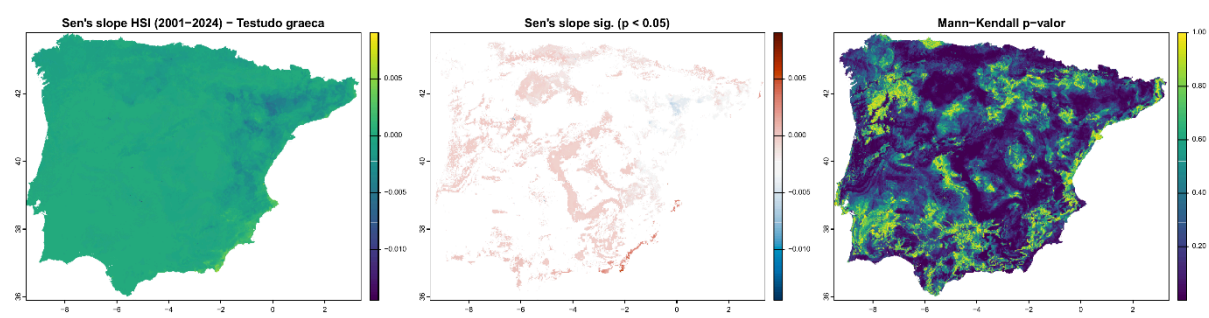


Figura A35. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Timon lepidus*.

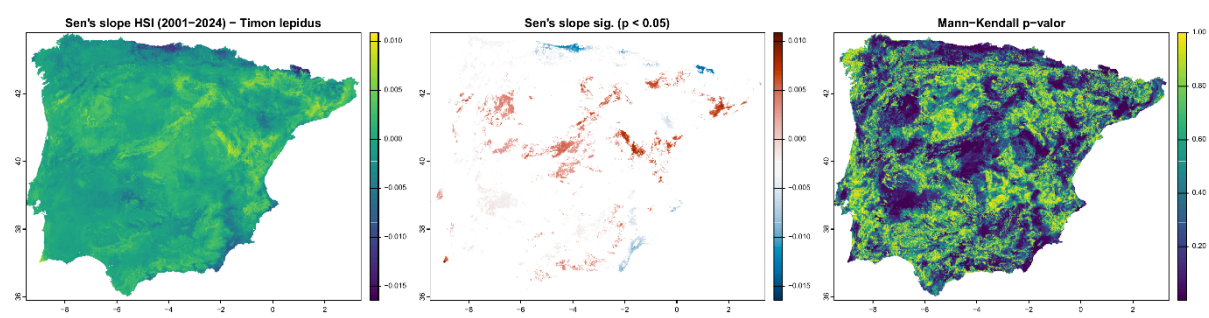


Figura A36. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Vipera latastei*.

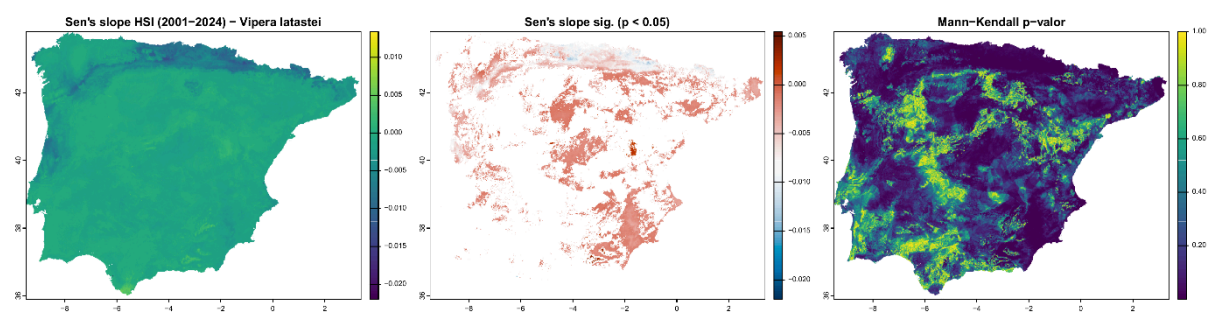


Figura A37. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Vipera seoanei*.

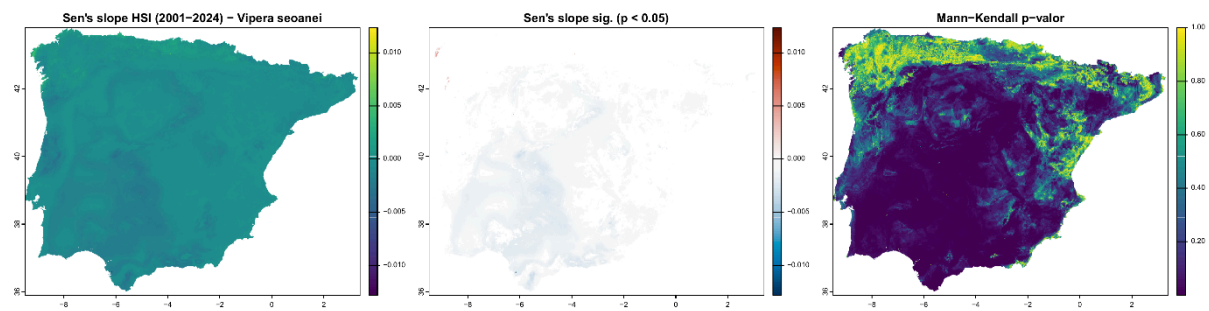
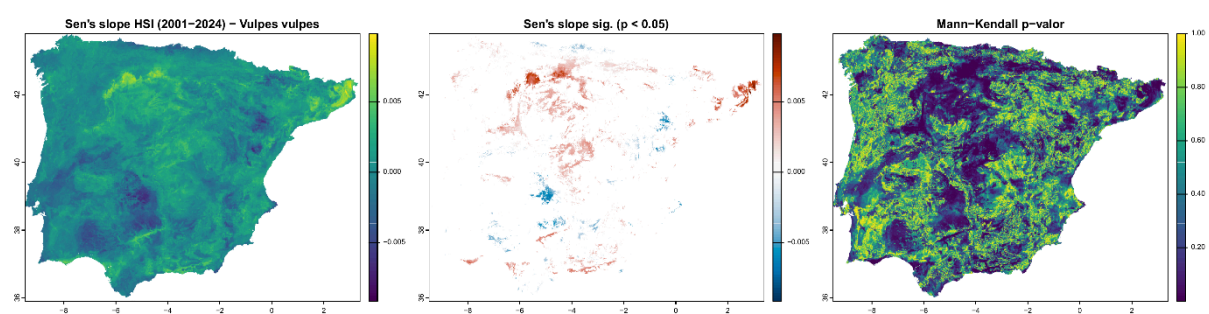


Figura A38. Tendencia temporal de la idoneidad del hábitat (Sen's slope y significancia de Mann-Kendall) para *Vulpes vulpes*.



Anexo XV: Curvas de respuesta de las variables ambientales en los modelos de ensamble para las 38 especies estudiadas.

Figura B1. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Abies pinsapo*.

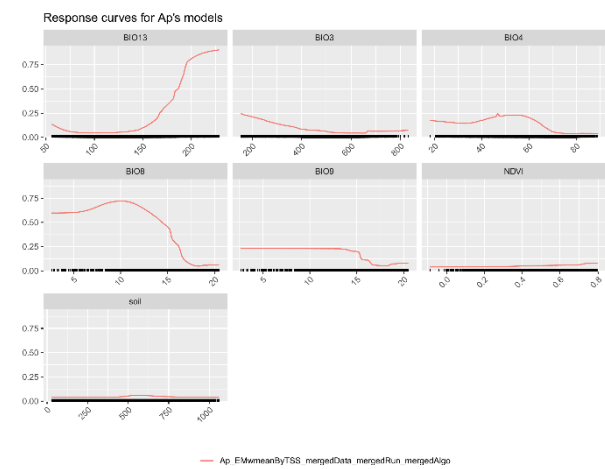


Figura B2. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Aegypius monachus*.

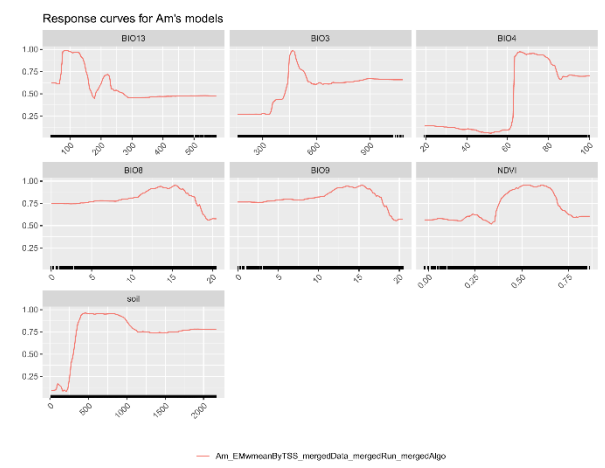


Figura B3. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Algyroides hidalgoi*.

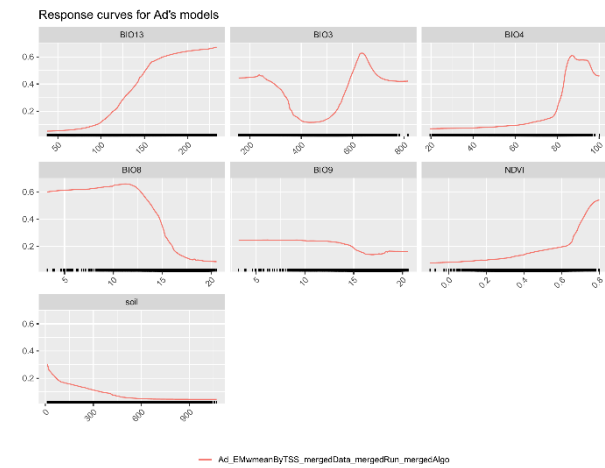


Figura B4. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Alytes cisternasii*.

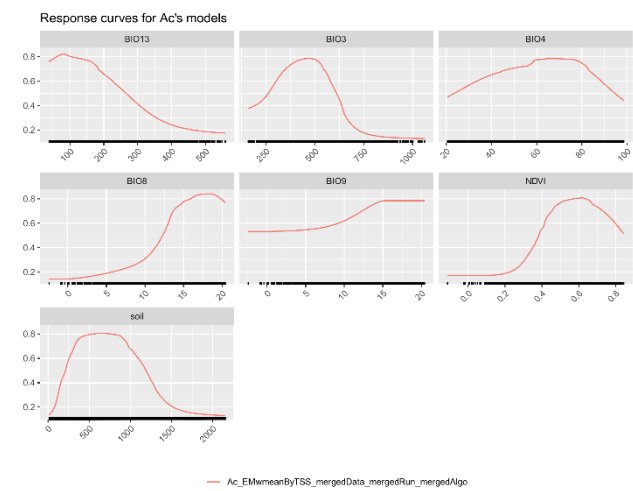


Figura B5. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Alytes dickhilleni*.

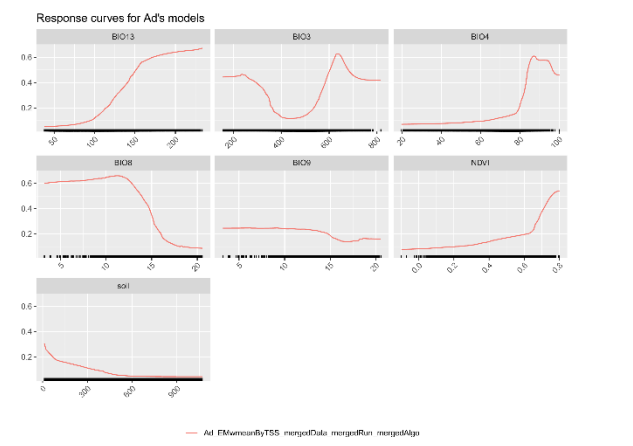


Figura B6. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Aquila adalberti*.

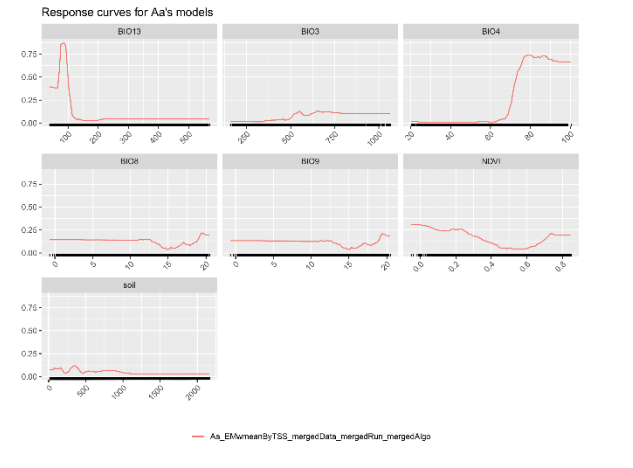


Figura B7. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Arbutus unedo*.

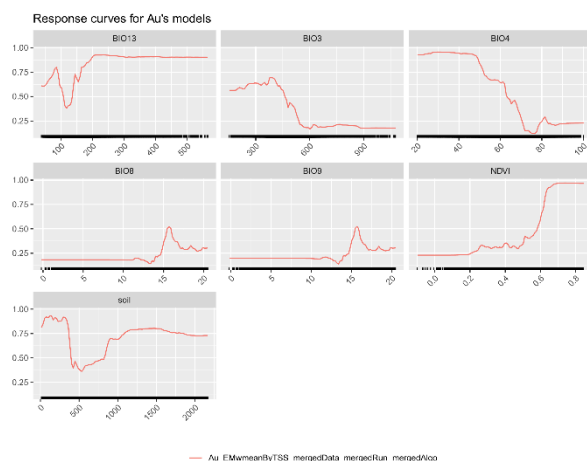


Figura B8. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Chersophilus duponti*.

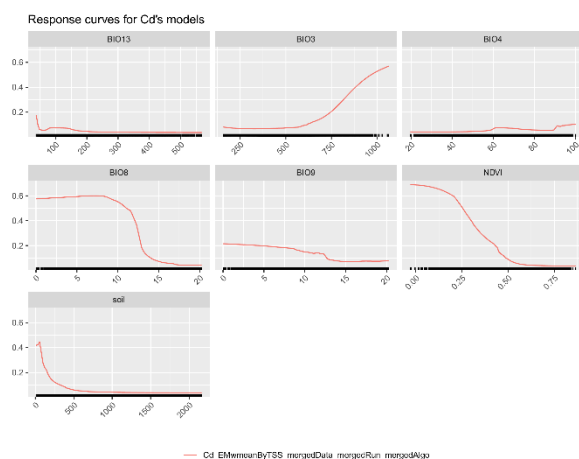


Figura B9. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Chioglossa lusitanica*.

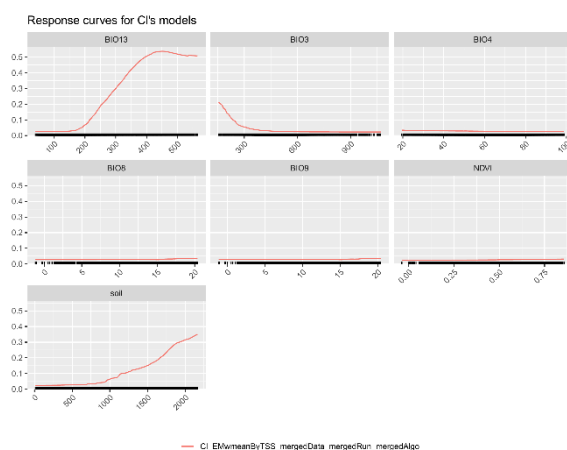


Figura B10. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Galemys pyrenaicus*.

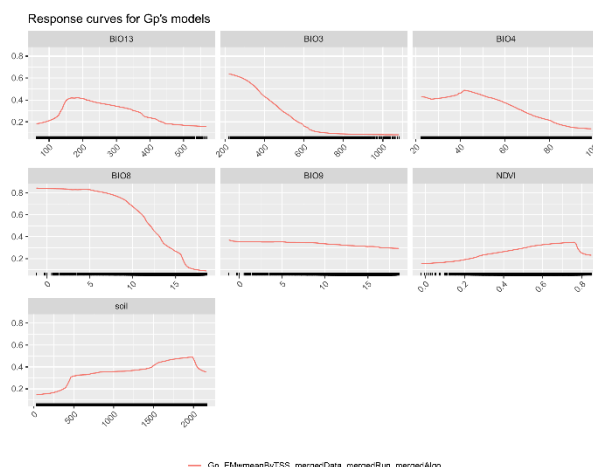


Figura B11. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Gypaetus barbatus*.

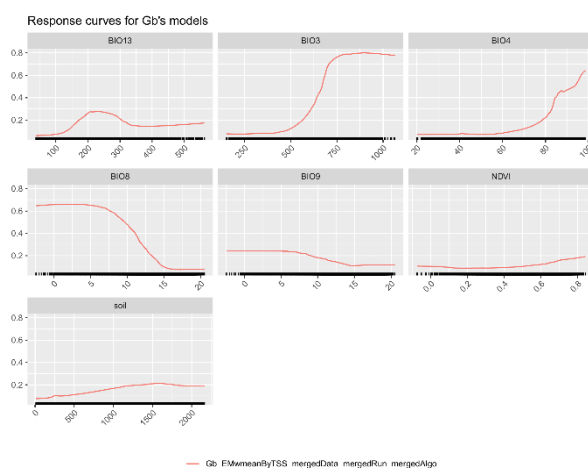


Figura B12. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Hyla meridionalis*.

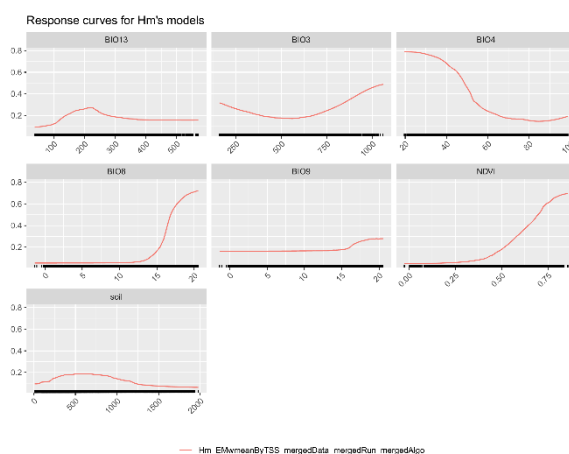


Figura B13. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Juncus heterophyllus*.

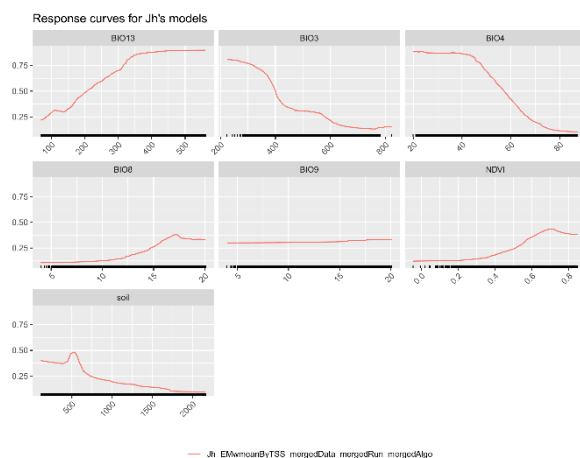


Figura B14. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Lanius senator*.

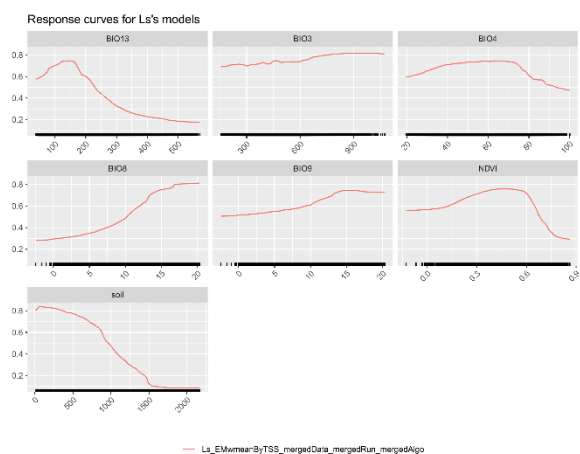


Figura B15. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Linaria ricardoi*.

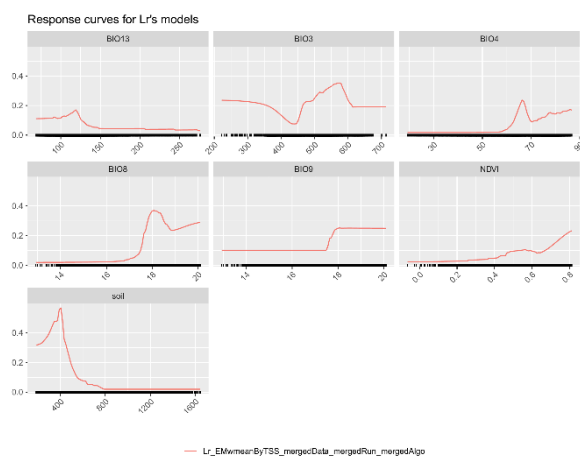


Figura B16. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Lutra lutra*.

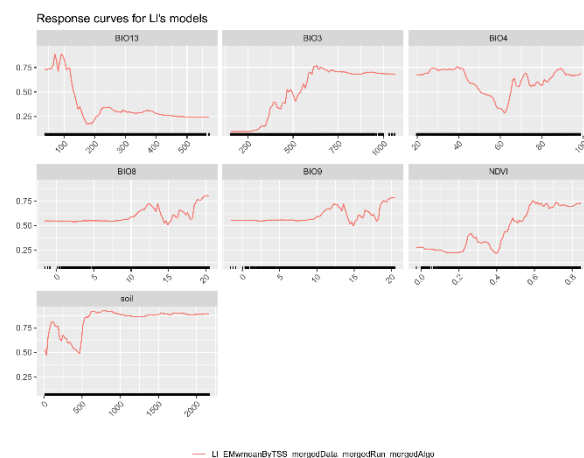


Figura B17. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Lynx pardinus*.

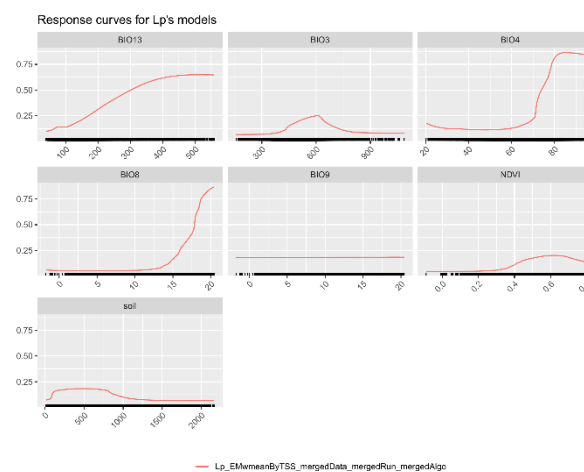


Figura B18. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Meles meles*.

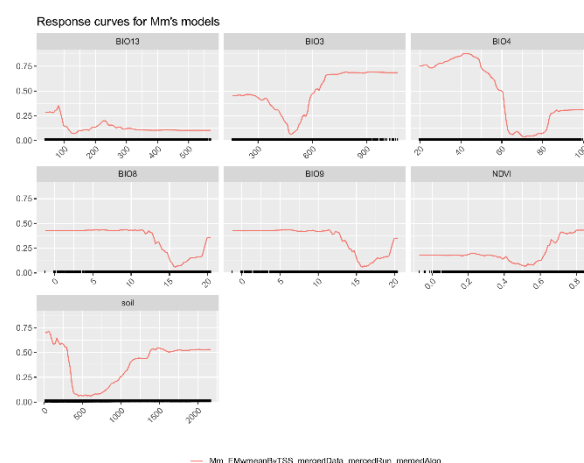


Figura B19. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Mustela lutreola*.

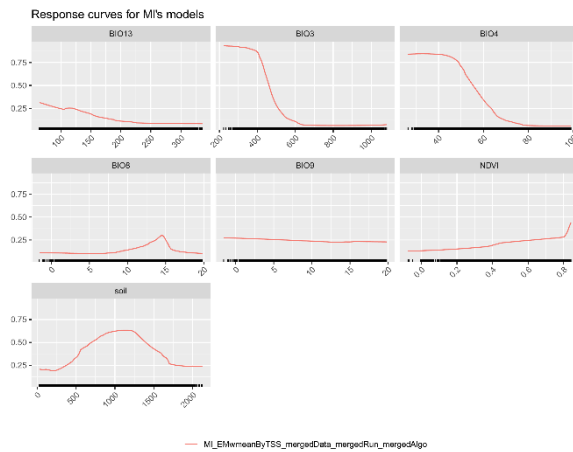


Figura B20. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Myotis capaccinii*.

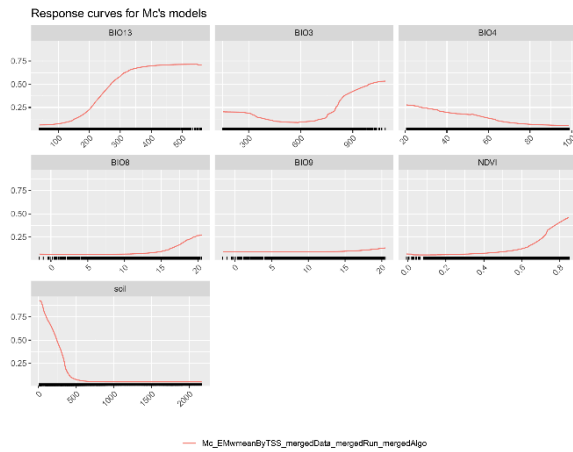


Figura B21. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Narcissus nevadensis*.

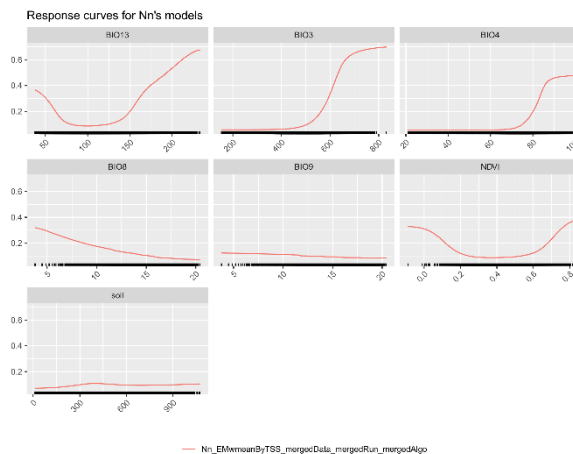


Figura B22. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Natrix maura*.

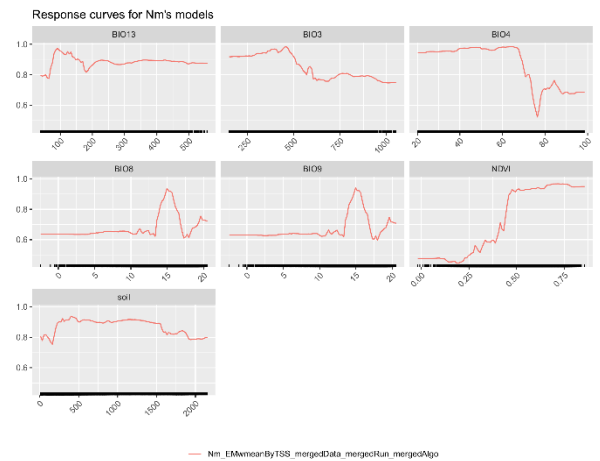


Figura B23. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Nyctalus lasiopterus*.

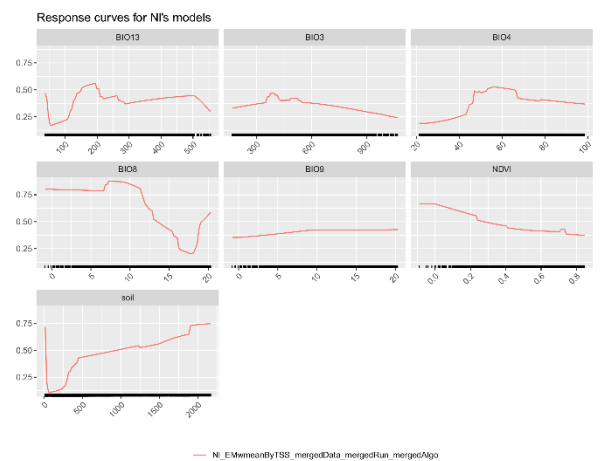


Figura B24. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Onobrychis viciifolia*.

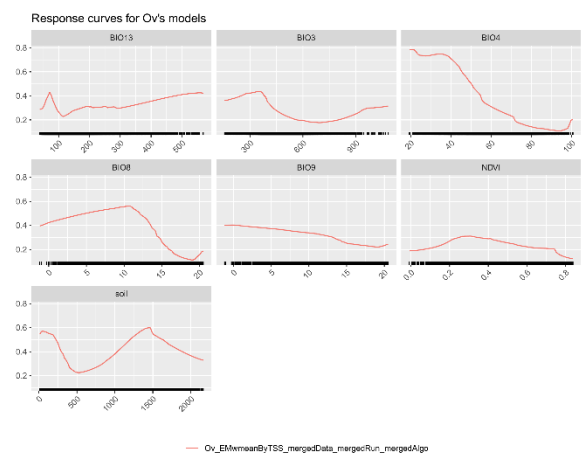


Figura B25. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Otis tarda*.

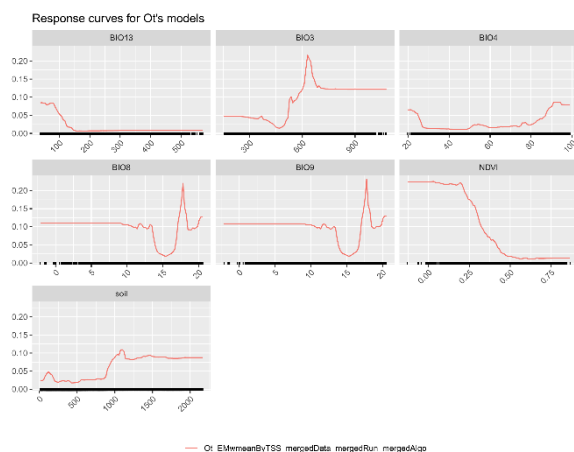


Figura B26. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Pelobates cultripes*.

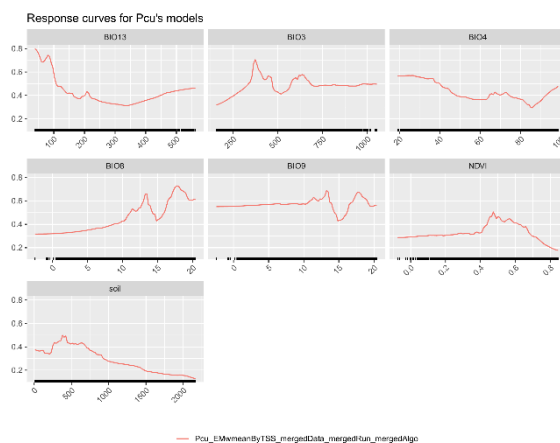


Figura B27. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Petrocoptis grandiflora*.

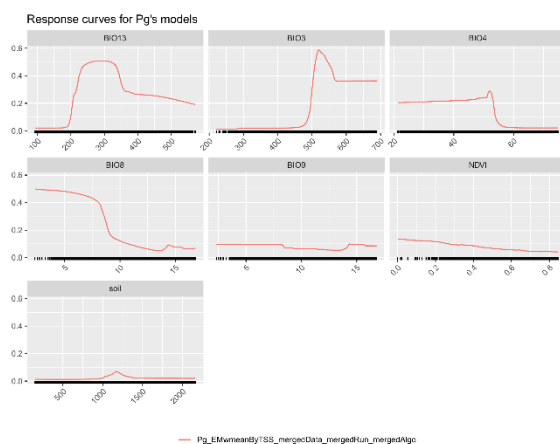


Figura B28. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Pleurodeles waltl*.

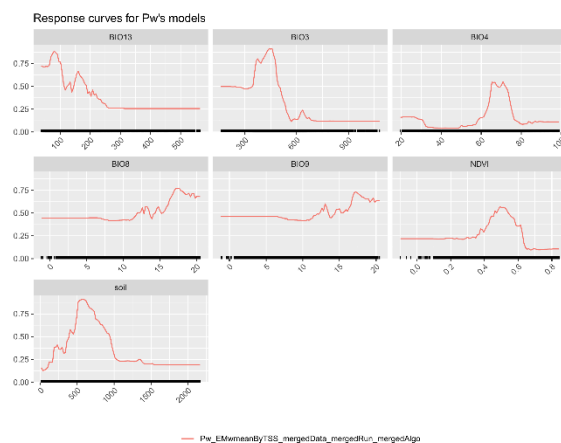


Figura B29. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Podarcis carbonelli*.

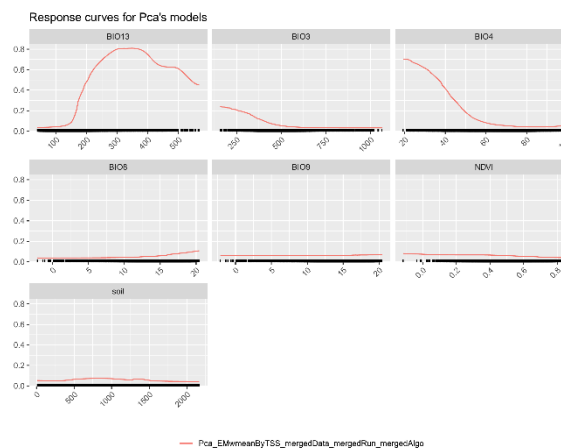


Figura B30. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Prunus lusitanica*.

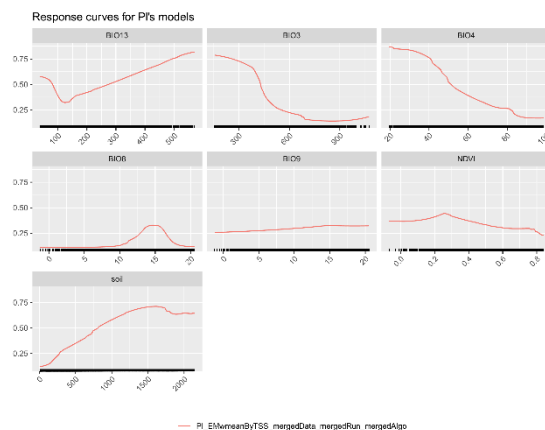


Figura B31. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Pterocles orientalis*.

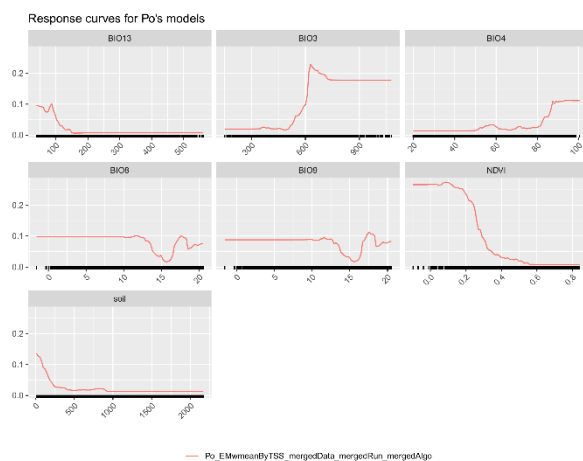


Figura B32. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Rana iberica*.

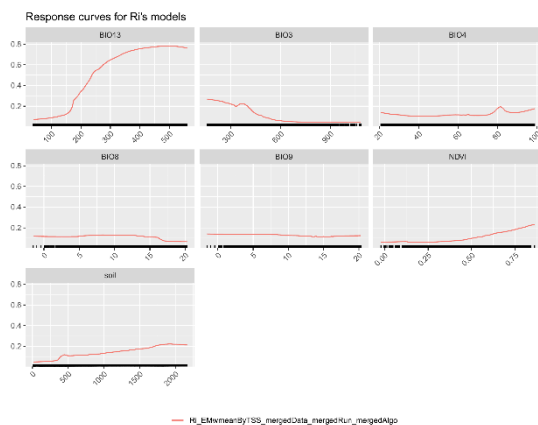


Figura B33. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Rana pyrenaica*.

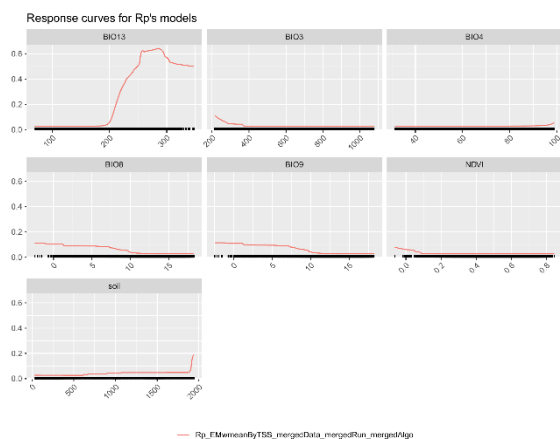


Figura B34. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Testudo graeca*.

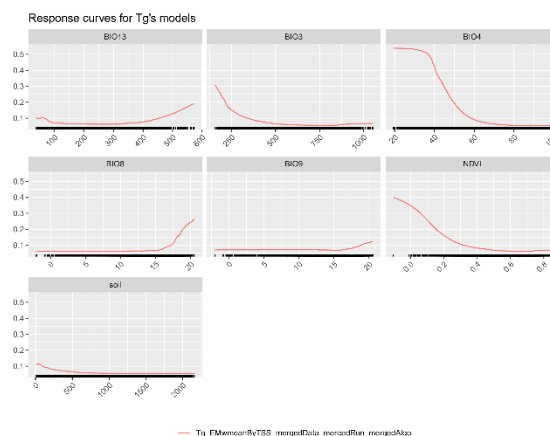


Figura B35. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Timon lepidus*.

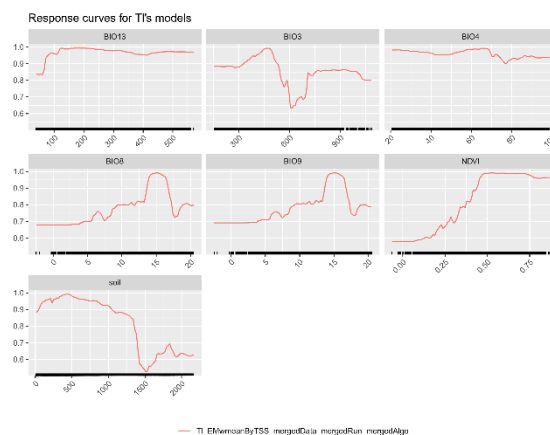


Figura B36. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Vipera latastei*.

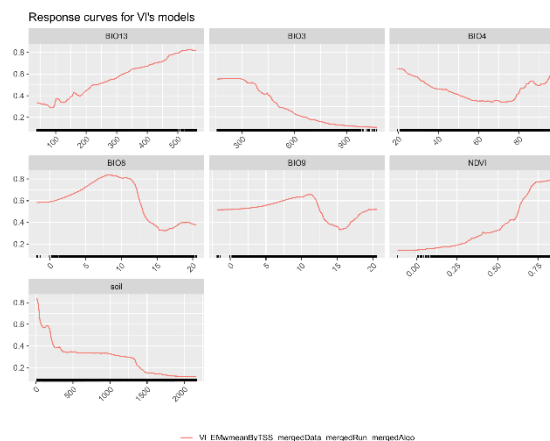


Figura B37. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Vipera seoanei*.

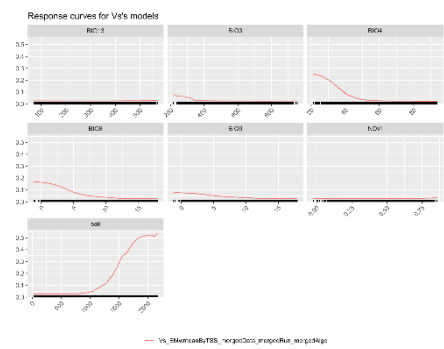


Figura B38. Curvas de respuesta de las variables ambientales en el modelo de ensamble para *Vulpes vulpes*.

